



OLLSCOIL NA GAILLIMHE
UNIVERSITY OF GALWAY

School of Engineering

Team 1 Project Report – Smart Soccer Ball System

Student/Group Name(s):		Student ID Number(s):
(1) Aftab Ahmmed, (2) Kento O'Connor		(1) 23344296, (2) 23481984
Discipline:	Electronic and Computer Engineering	
Subject Code and Name:	3 rd Year Project	
Supervisor Name:	Dr. Adnan Elahi	
Title of Report:	Smart Soccer Ball System	
Submission Deadline:	12/04/26	
Submission Date:	12/04/26	

Academic Integrity

Academic integrity is a fundamental value of the university community, based on a spirit of honesty. All students must read the University Academic Integrity Policy at www.universityofgalway.ie/academicintegrity/.

All work submitted by students for assessment purposes is accepted on the understanding that it is their own work and written in their own words, except where explicitly referenced using a correct format. Specifically, in the context of engineering, **students must NOT copy information, ideas, portions of text, data, images, figures, designs, CAD drawings, software code, or any other material, from any source without giving a complete reference to the source.** Sources include websites, other students' work, generative artificial intelligence (e.g ChatGPT, Google Gemini, CoPilot), books, journal articles, reports, etc. Students may not re-use work previously submitted to another course within the University or in another Institution.

Student Declaration on Academic Integrity

I have read and understood the University Academic Integrity Policy. I confirm that the content of this document is my own work, except where otherwise referenced in detail. I accept that breaches of academic integrity will be dealt with under the Process for Suspected Academic Misconduct as outlined in the Academic Integrity Policy.

Signature

Kento
Aftab Ahmmed

Table of Contents

1 – Abstract

1.1 Introduction

1.2 Background Research

1.2.1 Existing Technologies

1.2.2 Ultra-Wideband (UWB) Applications

1.2.3 Mathematical Approaches

2 – Project Management

2.1 Workload Allocation

2.2 Project Deliverables

2.3 Work Packages

3 – Technical Design

3.1 Initial Design

3.2 Overall System Design

3.3 Hardware Design

3.3.1 Anchor Hardware Design

3.3.2 Tag Hardware Design

3.3.3 Receiver Design

3.4 Software Design

3.4.1 Ball/Tag Code

3.4.2 Anchor Code

3.4.3 Receiver Code

3.4.4 Python Code

4 – Analysis & Results

4.1 Two-Way Ranging Calibration Experiment

5 – Conclusion

5.1 Expected Outcomes

5.2 Project Achievements

5.3 Challenges Faced

5.4 Recommendations for Future Work

Table of Contents

6 – References

7 – Appendix

7.1 Microcontroller Source Code

7.1.1 Anchor Source Code

7.1.2 Tag Source Code

7.1.3 Receiver Source Code

7.2 Python Monitor Source Code

7.2.1 main.py

7.2.2 config.py

7.2.3 kick_detector.py

7.2.4 trilaterate.py

7.2.5 position_filter.py

7.2.6 visualiser.py

7.2.7 visualiser_3d.py

7.2.8 simulate.py

1 Abstract

This project presents the design and implementation of a real-time soccer ball tracking system using Ultra-Wideband (UWB) technology. A UWB module and Inertial Measurement Unit (IMU) were embedded inside a foam soccer ball, communicating with a network of four fixed anchor nodes to determine the ball's position in 2D and 3D. Two-Way Ranging was used to measure distances between the ball and each anchor, with trilateration and Exponential Moving Average filtering applied on the laptop to produce a smoothed position estimate. Results demonstrated a mean positioning error of 4.2 cm and a maximum error of 9.1 cm at distances up to six metres, with the system operating at approximately 16 Hz. A live Python visualisation displayed the ball's 2D and 3D position, speed, and IMU data in real time.

The Smart Soccer Ball project explores how UWB technology can be used for sports tracking by placing a UWB module and an Inertial Measurement Unit (IMU) inside a soccer ball. Using a network of fixed anchor nodes, the system can track the ball's position in real time through ranging measurements, with the data displayed live through a Python interface.

The goals of this project are as follows:

- Implement 2D positioning using a minimum of three UWB anchors
- Extend the system to 3D positioning using a fourth anchor
- Integrate IMU sensors to capture accelerometer and gyroscope data alongside position
- Develop a real-time Python visualisation of ball position and movement

1.1 Introduction

Accurate real-time tracking has become increasingly important across many industries, particularly in sports analysis, healthcare monitoring, logistics, and security, where precise location and low-latency data are essential. While GPS is widely used for outdoor positioning, it typically provides accuracy only within several metres and suffers from latency issues, making it unsuitable for fast-moving, close-range applications. This is especially evident in sport, where centimetre-level precision and real-time responsiveness are critical. Ultra-Wideband (UWB) technology addresses these limitations by providing centimetre-level accuracy through time-of-flight ranging, making it a far more suitable solution for applications where precision is paramount.

1.2 Background Research

1.2.1 Existing Technologies

A range of wireless technologies can be used for indoor positioning, each offering different advantages and limitations in terms of accuracy, cost, and implementation complexity. GPS is widely available and effective outdoors, but because it depends on direct signals from satellites, its performance indoors is poor, and its accuracy is generally limited to several metres even under favourable conditions.

Wi-Fi fingerprinting and Bluetooth Low Energy (BLE) are both commonly used for indoor positioning, but they estimate distance using Received Signal Strength Indication (RSSI). RSSI-based methods are highly sensitive to multipath interference, where signals reflect from walls, ceilings, and other surfaces before reaching the receiver. These reflections can cause large fluctuations in signal strength, reducing accuracy and reliability. As a result, Wi-Fi and BLE systems generally provide positioning accuracy of around one to three metres, which is not suitable for sports tracking applications that require centimetre-level precision.

1.2.2 Ultra-Wideband (UWB) Applications

Ultra-Wideband (UWB) is a short-range wireless communication technology that operates by transmitting very short radio pulses across a very wide frequency spectrum, typically spanning 500 MHz. Unlike narrowband technologies such as Bluetooth or Wi-Fi, which estimate distance through received signal strength indicator (RSSI), UWB measures the precise time at which a radio pulse travels between two devices. This Time-of-Flight (ToF) approach makes UWB largely immune to the multipath interference and signal fading that degrade RSSI-based methods, resulting in positioning accuracy at the centimetre level under typical indoor conditions.

UWB works across a frequency range of 3.1–10.6 GHz, based on the IEEE 802.15.4a standards. The DW1000 module used in this project is one of the most used UWB chips in both research and industry. It allows different settings, such as data rate and channel selection, to be adjusted depending on whether the priority is longer range, lower power consumption, or faster position updates.

1.2.3 Mathematical Approaches

The ranging method used in this system is Two-Way Ranging (TWR), a technique that eliminates the need for synchronised clocks between devices. In TWR, the tag (the ball) initiates a transmission to an anchor at timestamp T1. The anchor receives this at T2, processes it, and replies at T3. The tag receives the reply at T4. Using these four timestamps, the round-trip time can be calculated while cancelling out processing delays, so the true time of flight is:

$$\text{ToF} = ((T4 - T1) - (T3 - T2)) / 2$$

Multiplying ToF by the speed of light gives the physical distance between the tag and that anchor. This computation is performed on the tag's ESP32 after each anchor forwards its T2 and T3 timestamps back, keeping the ranging logic consistent. With distances from multiple anchors measured, the ball's position is determined through trilateration. Each distance defines a sphere centred on a known anchor position with a radius equal to the measured distance. The ball is at the point where all the spheres intersect. However, in practice, the spheres do not meet exactly

because of noise, so the least-squares approach is used to find the position that minimises the total error across all distances at the same time.

Sphere equations are nonlinear so directly solving this system is computationally expensive and difficult. Therefore, a linearisation technique is applied. The distance equation for anchor one is subtracted from each of the remaining anchor equations, cancelling the squared position terms and converting the nonlinear system into a linear one. This linear system is then solved using NumPy's least-squares solver, giving the estimated 3D position of the ball. A minimum of three anchors is needed for 2D positioning and 4 anchors are needed for a definite 3D position. The spread of the anchors across the playing area is also very important. If the anchors were too close together, the Dilution of Precision (DOP) increases, which would mean that even small distance errors would cause much bigger errors in final position estimate. In this system, four anchors are placed at the corners of a rectangular cuboid, with two anchors placed at floor level and two elevated at the opposite corners, providing a sufficient geometric spread in all three axes to support stable 3D positioning and minimises DOP.

2 Project Management

2.1 Workload Allocation

The project was divided into three main sections:

- Research
- Electronics
- Programming

Each member was allocated a certain degree of responsibility for each section and subsection of the project as seen in the tables below.

Research/Deliverables	Aftab Ahmmed	Kento O'Connor
Component Research	70%	30%
Existing Methods Research	20%	80%
System Architecture	60%	40%
Weekly Reports	50%	50%
Project Poster	50%	50%
Report Writing	50%	50%

Electronics/Hardware	Aftab Ahmmed	Kento O'Connor
Circuit Design	60%	40%
Wiring	50%	50%
Soldering	50%	50%
UWB Calibration	40%	60%
Housing CAD Design	80%	20%
Ball Construction	40%	60%

Software	Aftab Ahmmed	Kento O'Connor
Anchor	60%	40%
Tag	70%	30%
Receiver	70%	30%
Python Trilateration	30%	70%
Python Graphics	100%	-
Python Filtering	40%	60%

2.2 Project Deliverables

Fixed deliverables were allocated for this project. These were mandatory tasks needed for a valid and successful project. The fixed deliverables were as follows:

- Project Plan – A document which describes the project’s work break down
- Progress Presentation – A presentation at the half-way point of the project’s timeline
- Project Dissemination – A poster describing the project’s aims and objectives
- Bench Demonstration – An in-person demonstration, presenting the project at the end of the semester to supervisors.
- Final Project Report – Report documenting all steps and tasks involved in reaching the point of submission for the project.

2.3 Work Packages

The team divided the progression of the project into “blocks” called work packages. Each work package had set tasks and an allocated completion date.

	To summarise; The workload approach is: • Research • Design • Construct • Get the minimum implementation working • Run test and analyse results • Reiterate and improve • And to repeat this for every feature of the project. WP#1: Research into triangulation code, ball structure, hardware housing and IMU coding. WP#2: Designing process for ball internal hardware housing and ball structure. Familiarising with ESP32 hardware. WP#3: Construction of ball’s housing and structure. Familiarising with UWB technology WP#4: Testing of project thus far. Construction UWB anchor housing design. Refining based on said testing. WP#5: Commencing of positional algorithm coding. Improving output display being shown to user. Testing of triangulation code. Refining code based on testing with implemented positional tracking.
--	---

	WP#6: Using additional/supplementary software to display a 3D graphic of the balls position on pitch. Adding user settings for different playing scenarios. Overall quality control of necessary project aspects.
--	---

Work Package #1	Background Research/Fundamental Techniques
Task #1	Research code and calculations for IMU value derivations
Task #2	Research necessary smoothing/filtering/triangulation algorithms
Task #3	Research ball internals structures and materials
Linked Deliverables	Weekly Progress Report
Expected Outputs	Near complete knowledge of implementation method

Work Package #2	Feasibility testing/Minimum implementation
Task #1	Design of internal ball structure
Task #2	Design of internal hardware housing
Task #3	IMU calibration and reading output
Task #4	Establish communication between two ESP32 MCs
Linked Deliverables	Weekly Progress Report
Expected Outputs	Finalised ball hardware design, IMU ready for testing

Work Package #3	Core Project Implementation 1
Task #1	Simple position linear tracking between 1 anchor and tag
Task #2	Construction of ball hardware housing
Task #3	Design anchor/antenna hardware housing setup
Task #4	Integration of hardware housing into ball's structure
Linked Deliverables	Weekly Progress Report, Project Progress Report + Presentation
Expected Outputs	Complete basic implementation – Ball with accel. ready to test.

Work Package #4	Core Project Implementation 2
Task #1	Testing of ball with accelerometer
Task #2	Construction of anchor/antenna module housing
Task #3	Refining of base implementation based on testing
Linked Deliverables	Weekly Progress Report, Project Progress Report + Presentation
Expected Outputs	Anchor housing, User output from accelerometer,

Work Package #5	Core Project Implementation 3
Task #1	Begin code implementation with multiple anchors
Task #2	Start interface design of user statistics being displayed
Task #3	Testing of trilateration techniques (2D Mapping)
Task #4	Refining and implementation of mobile tag in ball testing
Linked Deliverables	Weekly Progress Report, Project Progress Report + Presentation
Expected Outputs	Majority of Project achieved

Work Package #6	Enhancements & Additional Achievements
Task #1	Create live 3D graphic of ball's position
Task #2	Implement different ball setting modes for different scenarios
Task #3	Refining previous features as necessary
Linked Deliverables	Weekly Progress Report, Project Dissemination (poster/video/web page/social media/reflective essay) Bench Demonstration & Interview (demonstrator)
Expected Outputs	Optimised user experience, 3D graphical user interface, Specific ball feedback based on selected scenarios.

3 Technical Design

3.1 Initial Design

One of the main challenges at the start of the project was figuring out how to physically house the electronics inside a soccer ball without compromising the structural integrity of the ball or the playability of the system. Several approaches were explored before a final solution was reached.

The first approach that was considered was to remove the bladder entirely and inserting a rigid electronics module inside the ball, held in place using an internal strap system. This concept was inspired by commercial smart balls such as the Adidas miCoach, where a central sensor unit is suspended using straps attached to the inner surface.



Figure 3.1: Adidas miCoach Smart Ball [2]

While this approach would preserve the external look and feel of a normal soccer ball, it introduced several practical challenges. Fitting all the required hardware into the limited internal space without affecting the shape of the ball proved very difficult. Re-sealing the ball after installing the electronics was also problematic, as it would need to remain airtight while still allowing for wireless communication and some form of charging access.

Another approach explored was embedding the electronics between the layers of the soccer ball. This involved cutting open one of the panel seams to access the interior and placing the hardware between the outer shell and the inner bladder. However, this would create a noticeable protrusion where the sensors were located, affecting the spherical symmetry of the ball and causing it to roll unevenly. In addition, placing the electronics so close to the outer surface increased the risk of damage during regular use.

Considering these challenges, it was decided that instead of trying to force the electronics into a standard soccer ball, the design of the ball itself would be adapted. An indoor foam ball was chosen as the base for the prototype. The foam material made it easy to cut the ball cleanly in half, creating two equal sections with a cavity in the centre. Most importantly, this eliminated the risk of air leaks, as we no longer had to cut open and re-seal a pressurised bladder.

3.2 Overall System Design

The overall system design of the project involves a cuboidal 3D playing area encompassed by four UWB modules set to “anchors” on each corner. A ball with an embedded UWB module acts as a “tag”, which communicates with a set of fixed anchors placed around the playing area. It sends signals to each anchor one by one measuring the time of flight of the UWB pulses to derive a distance estimate to each anchor. These distance measurements are then sent to a laptop, where a trilateration algorithm is used to calculate the ball’s position in three-dimensional space in real time.

The four anchors are placed around the space in a cuboid layout, with two positioned higher up and two at ground level. A well-spread cuboid arrangement ensures the system is not degenerate in any axis, allowing reliable 3D position.

Communication between the ball and the rest of the system is handled via two protocols operating in parallel. The DW1000 UWB modules handle the ranging exchanges directly between the tag and each anchor using Two-Way Ranging (TWR). Once all four distances have been computed, the ball transmits the ranging results along with the raw IMU data, (three axes of accelerometer and three axes of gyroscope) to a standalone base station ESP32 via ESP-NOW, a connectionless peer-to-peer wireless protocol. The base station is physically separate from the four anchors and connects to the laptop via USB. It acts as a bridge, forwarding the data it receives from the tag to the Python monitoring software. Since the laptop cannot directly receive ESP-NOW signals, this extra step is necessary for the system to work.

On the host laptop, a Python application receives the incoming data stream in CSV format and processes it in real time. The distance measurements are first smoothed using an Exponential Moving Average filter, then used in a trilateration algorithm to calculate the ball’s 3D position. The result is displayed live using PyQtGraph. Overall, the system runs at around 15-20 Hz, from initial ranging on the ball to the final position shown on screen.

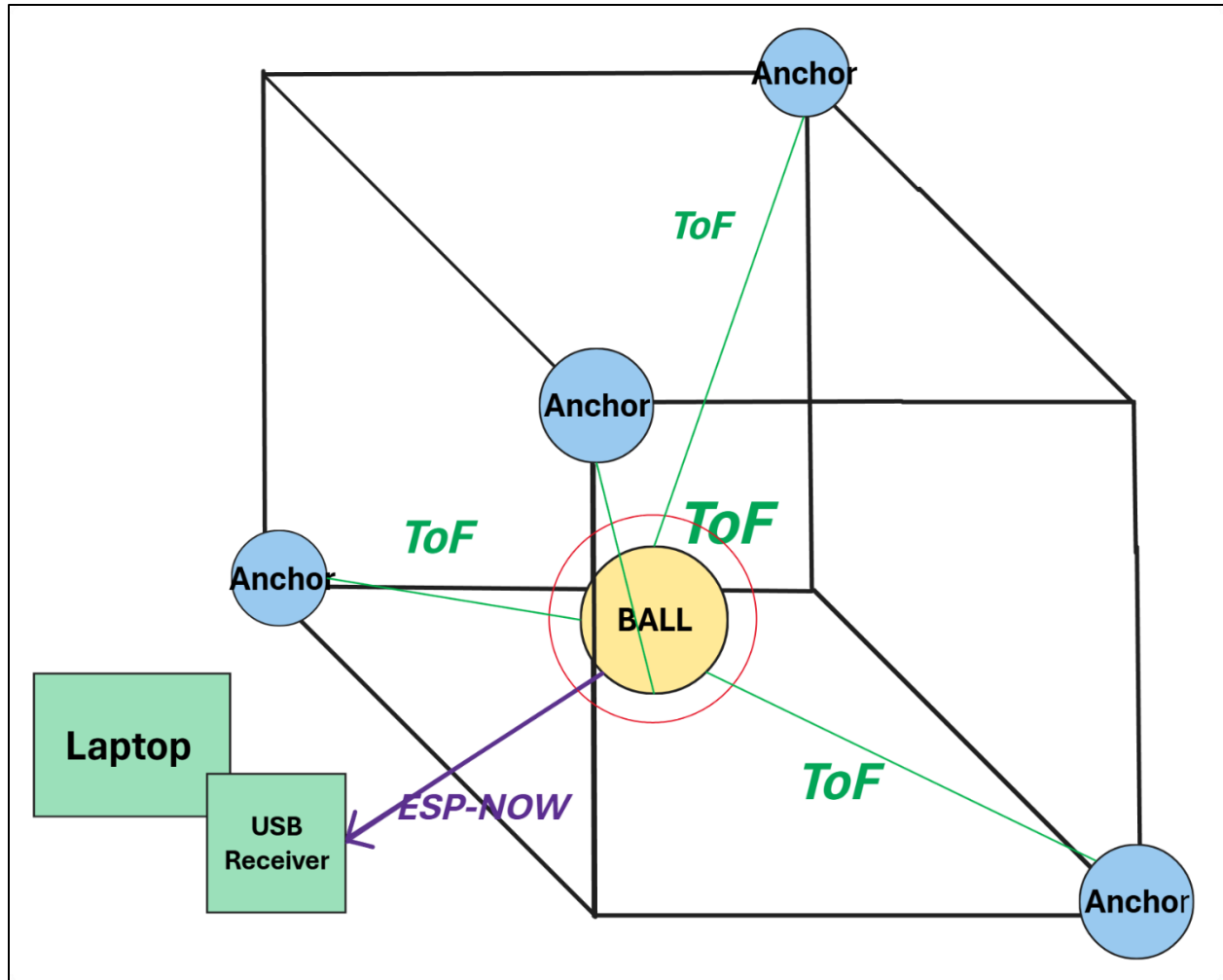


Figure 3.2: Diagram of Overall System

3.3 – Hardware Design

Circuits and housing units had to be constructed for the tag (ball) and anchors. The ball's internal structure required that all enclosed electronics were sufficiently protected from high-force impacts. The anchor's structures required that the antenna of the UWB module maintains as clear of a line of sight with the tag as possible while housing the enclosed electronics securely.

3.3.1 – Anchor Hardware Design

For the electronics of the anchor hardware the following parts were chosen:

- Microcontroller: The FireBeetle ESP32 v4
- Power source: Rechargeable 2000mAh 3.7V lithium-ion battery
- Sensor: Mikroe DW1000 UWB click module

This specific ESP32 module was chosen as it has a built-in voltage regulator for 3.7V to 3.3V conversion and battery port connector which allowed for battery charging via USB connection or supplying power.

The components were then soldered to a solder board, wired and then powered by the battery. The UWB module was wired to the ESP32 via SPI:

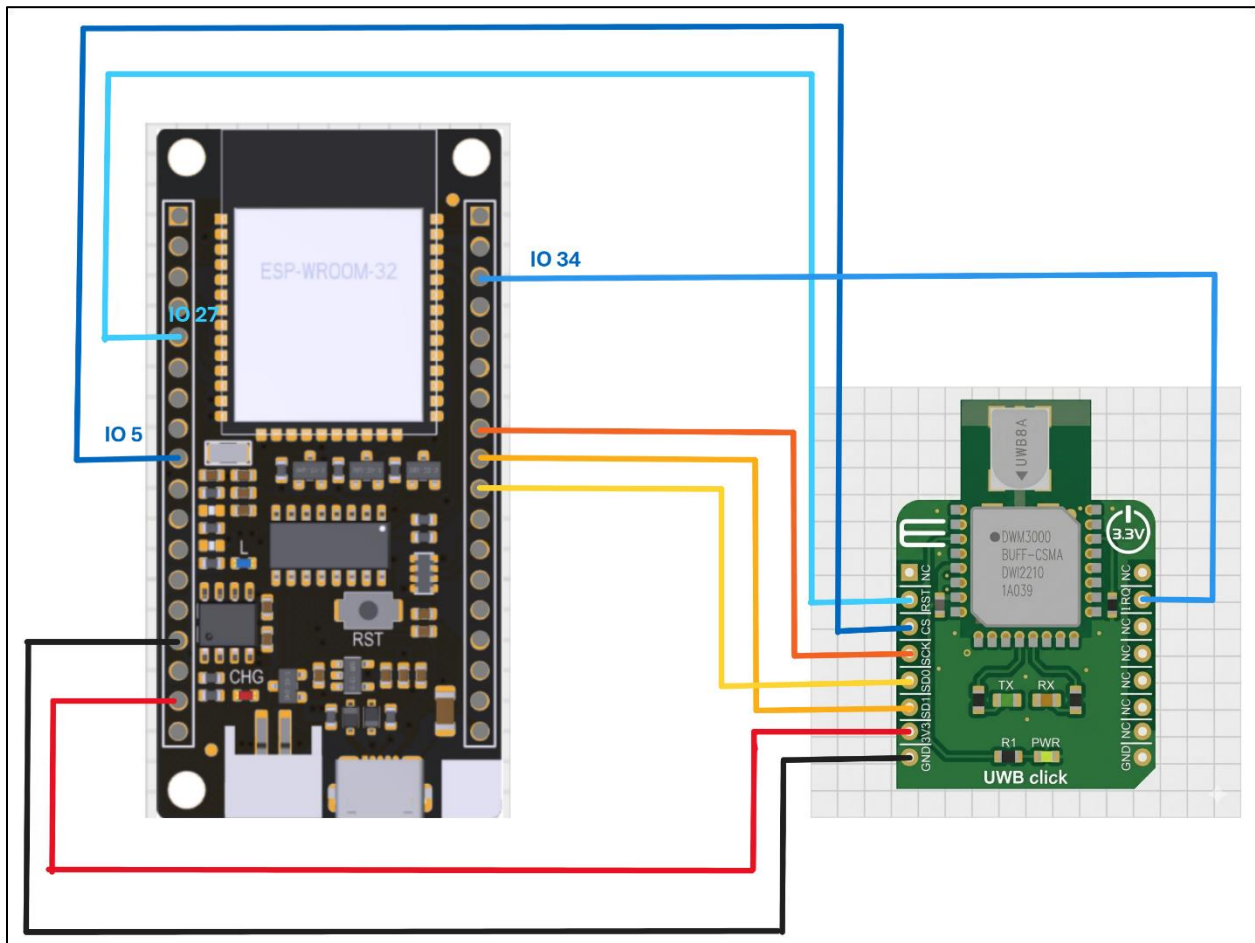


Figure 3.3.1: Circuit Diagram of Anchor

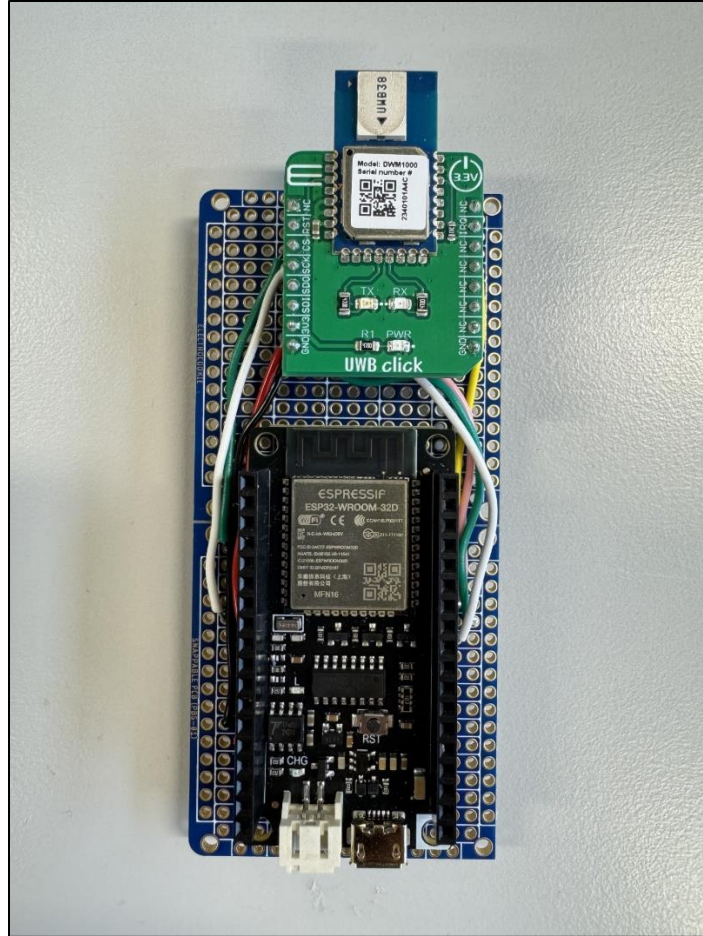


Figure 3.3.2: Soldered Board for Anchor

Housing for the anchor's electronics was then drawn in TinkerCAD and 3D printed once the circuit was completed. This housing had the circuit in an upright position within and enclosed insert with an opening for the UWB antenna to minimise interference. It had a pocket in the back for the battery. A small gap was made at the underside of the base of the anchor to allow for connection of the battery and a USB cable for charging and/or power.

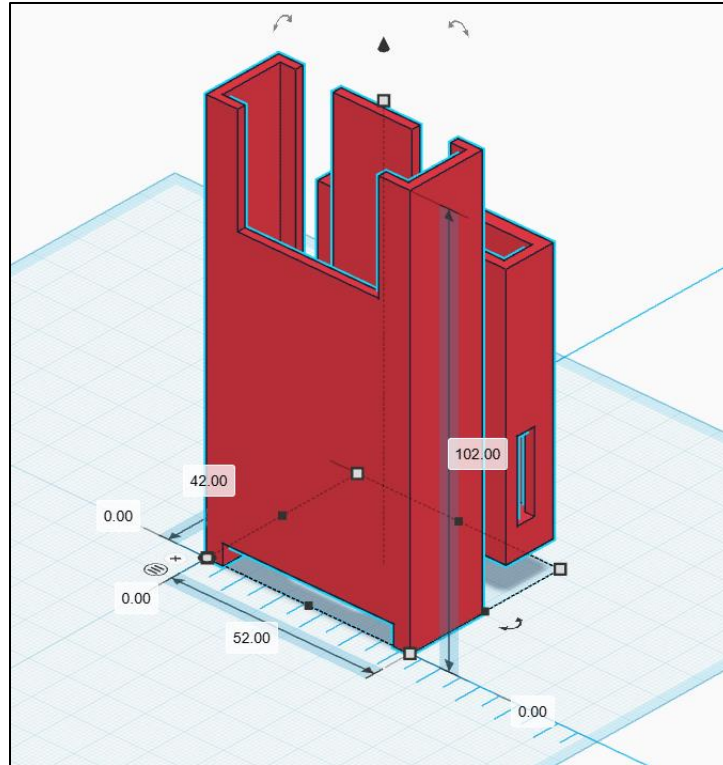


Figure 3.4: Tinkercad drawing of anchor housing

The CAD drawings were then printed for construction of four anchors units. The soldered circuits were placed within the housings and subtly mounted with hot glue on each corner of the board. The LiPo batteries were inserted into the rear slot and connected to the circuit via the port opening on the underside of the anchor housing. Thus, the four anchor modules required were constructed.

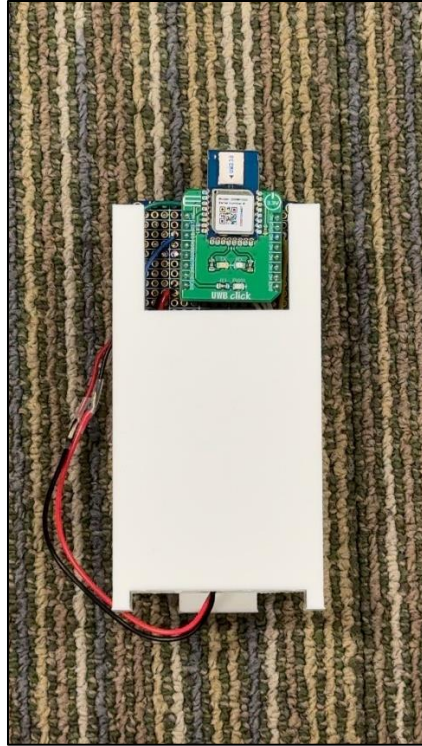


Figure 3.5: Fully Constructed Anchor

3.3.2 Tag Hardware Design

The ball required a hardware enclosure containing embedded sensors securely fitted in the core. This enclosure had to possess a small profile to not significantly interfere with playability. The ball would also have to be modified in manner that allowed ease of access to its contents for troubleshooting and battery charging.

For the electronics of the ball, the following components were selected:

- Microcontroller: The Adafruit Huzzah32 Feather
- Power source: Rechargeable 1800mAh 3.7V lithium-polymer battery
- Sensors: Mikroe DW1000 UWB click module, Mikroe 6DOF IMU 7 click

This specific microcontroller was chosen as it contains the necessary battery power and charging support like the anchor but in a smaller form factor allowing the physical footprint within the ball to be minimal. The smaller LiPo battery was chosen for the same reason. The IMU was chosen to allow data from the module's accelerometer and gyroscope to be read and analysed.

The wiring diagram of the ball's internal electronics was then designed. The UWB module was to be wired via SPI (Serial Peripheral Interface) of the microcontroller. The IMU module was to be wired to the microcontroller via I2C serial communication protocol. The microcontroller would remain powered by the 3.7V LiPo battery during testing.

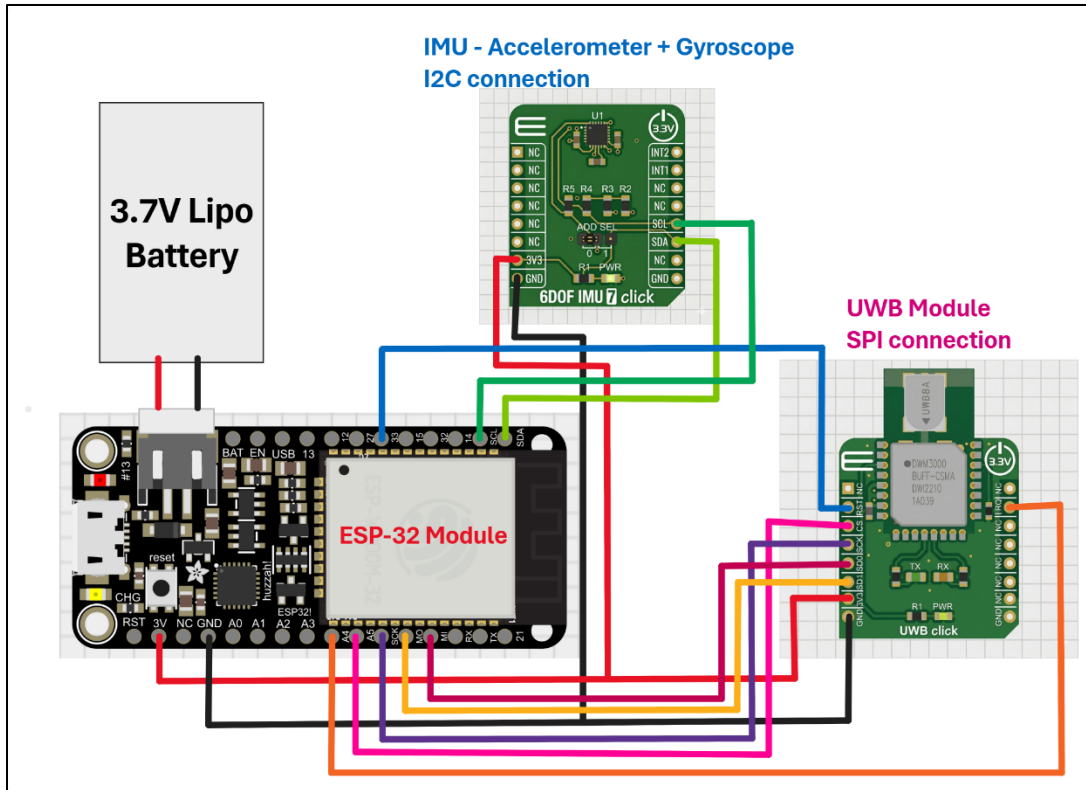


Figure 3.6: Circuit Diagram of Ball Acting as Tag

Electronics Layout and Wiring

The physical circuit was required to fit into the cube like shape of the internal ball housing, so the design's shape of the circuit reflected this. The design had two solderable boards stacked on top of each other with the ESP32 soldered between the two layers at a 90-degree angle. Female headers were soldered onto the top and bottom layers which allowed for detachable components. The IMU was mounted to the top layer and UWB module was mounted at the bottom. This resulted in the layout below:

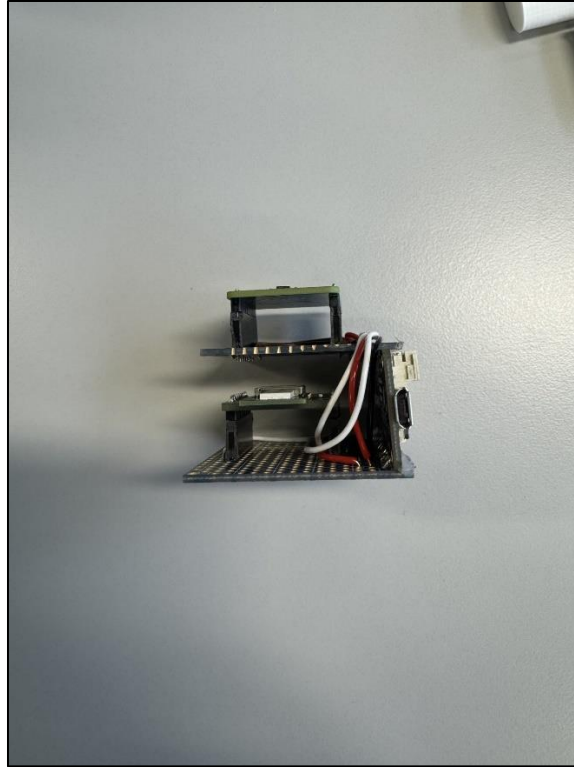


Figure 3.7: Image of Ball Wiring Setup

Ball Construction

The foam indoor ball selected for the project was modified to allow easy access to the core of the ball whenever hardware or housing needed to be altered. This was done by cutting the ball in half, forming two semi-circles. A cube was cut into the center of the flat face of each semi-circle. The depth of the cuts was approximately half the height of the hardware housing while the width and height matched the housing measurements. Velcro straps were attached and then glued to these faces. This allowed the two sphere halves to be joined securely while containing the embedded sensors within the core. The Velcro allowed the two halves to be separated when required for testing, charging or powering.

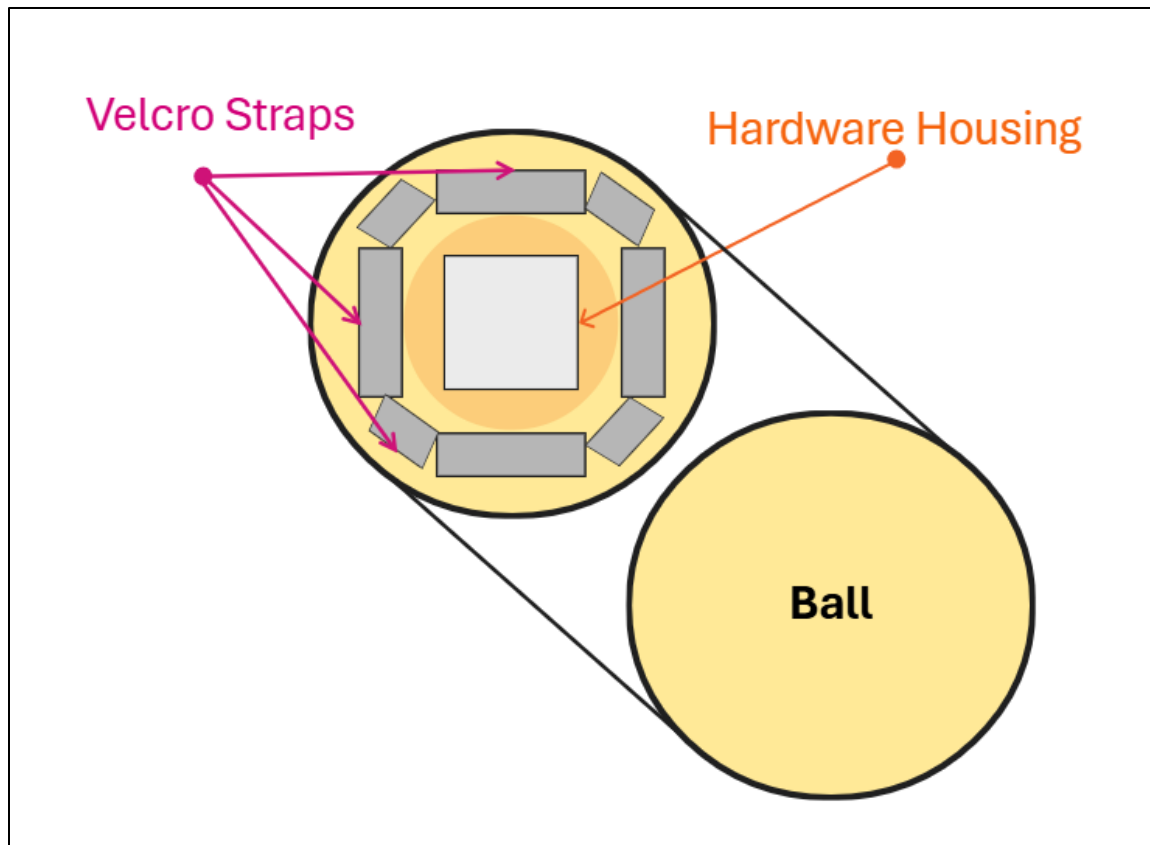


Figure 3.8: Diagram of Ball Internal Enclosure

Tag Hardware Housing

Housing for the ball components had to be developed and made. This housing would have to meet the following requirements:

- Securely house all electronics so that they do not move freely within the ball.
- Protect components (most importantly the battery) from high impact kicks.
- Have a minimal physical footprint so that the playability is not hindered.
- Allows the battery to be easily plugged in and out of the microcontroller.
- Allows a USB connection to be easily made to view microcontroller serial monitor and to charge battery.
- Not to significantly interfere with signals such as WiFi or UWB

The housing was designed in TinkerCAD while adhering to all these requirements. The final design was a cube-like shape with a removable lid.

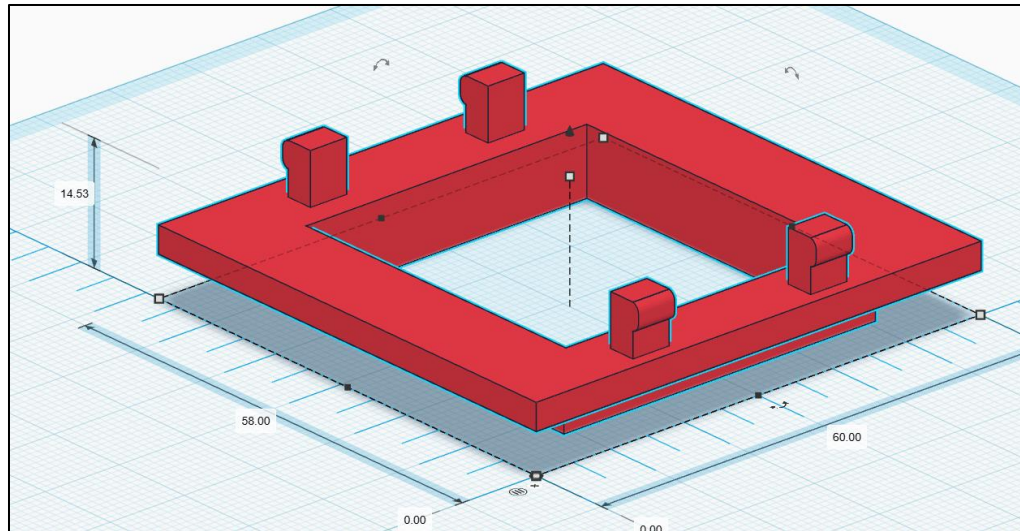


Figure 3.9: Lid of hardware housing shell

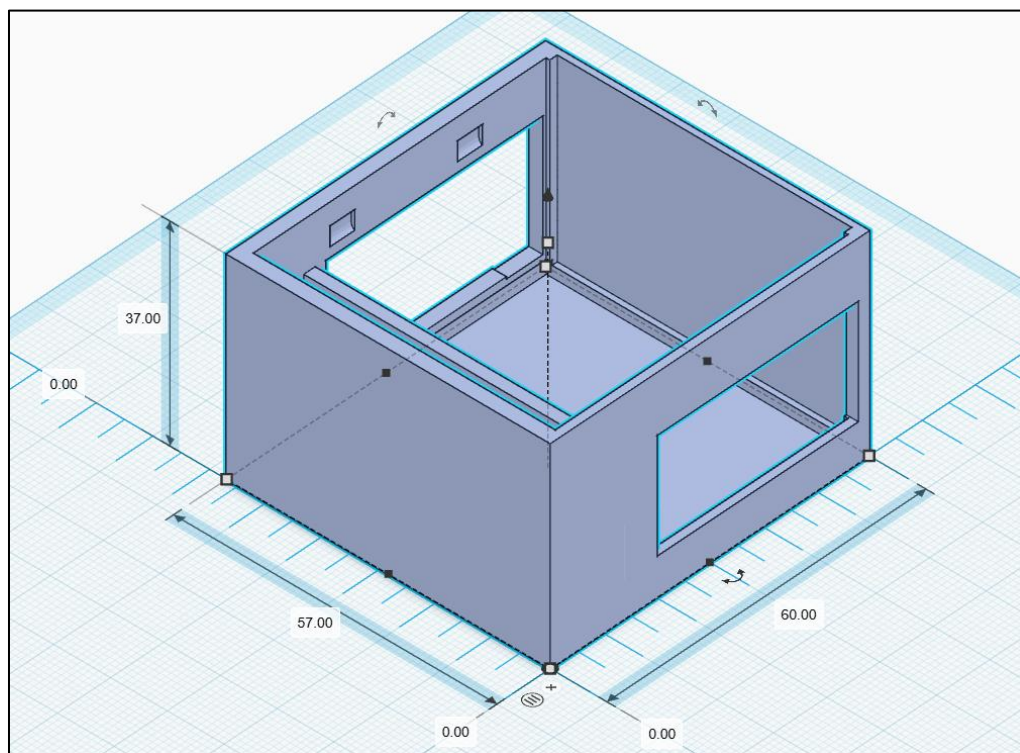


Figure 3.10: Tinkercad drawing of hardware housing main shell

A side pocket was made acting as a “sleeve” for the LiPo battery, giving it the necessary protection needed for use as a soccer ball. The main chamber was made to contain the microcontroller circuit. Windows were made on the sides to reduce weight from material and reduce any potential reflection/interference of UWB or Wi-Fi waves. The top lid was created for additional protection of the electronics while allowing temporary access to the inside of the housing.

The ball hardware was then fully constructed once the housing was 3D printed.

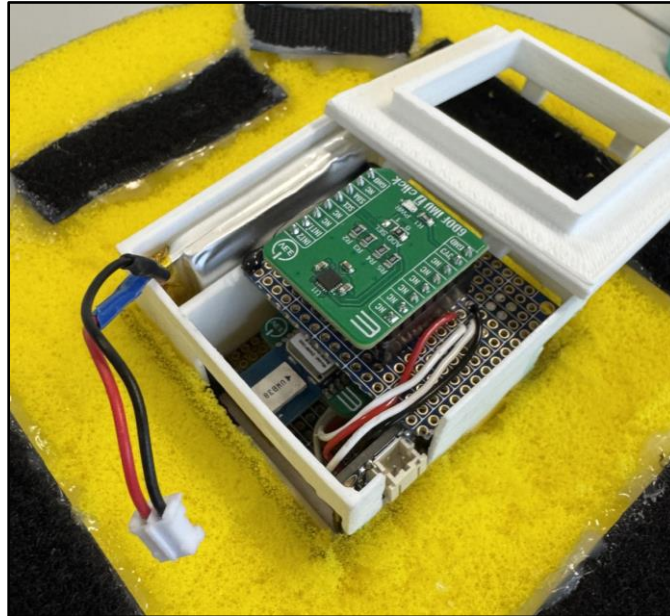


Figure 3.11: Completed Construction of Ball

3.3.3 Receiver Hardware Design

An Arduino ESP32 Nano was used as an ESP-Now receiver for communication with the ball's internal ESP32. This was connected to a laptop using a USB-C data cable.

3.4 Software Design

The soccer ball system relies on four different code spaces:

1. Anchor microcontroller Arduino code for ranging between each anchor and the one tag.
2. Ball microcontroller Arduino code for retrieving all ranges to each anchor and retrieving IMU values.
3. Receiver microcontroller Arduino code to receive CSV values from the ball via ESP-Now
4. Python tracking code to process CSV values from receiver, run calculations and filtering and displaying live 2D/3D graphics and plots.

3.4.1 Ball/Tag Code

This ESP32 Arduino code sets the UWB module to a mobile tag and continuously collects data from the UWB anchors and the IMU. The IMU is polled on a fixed timer, UWB ranging triggers callbacks when data arrives, and packets are sent only when fresh, complete data is available and rate limits allow.

At startup in the `setup()` function the ESP32 is set into Wi-Fi station mode and ESP-Now is initialised. A single peer (the receiver) is registered using its MAC address. The IMU is initialized over the I2C protocol, and the UWB module is initialized over SPI. The UWB library

is configured in tag mode and callback functions are registered so the program is aware whenever a new UWB range is measured or an anchor appears or disappears.

The main loop does very little work directly. Every 10 ms it reads the IMU, storing the latest acceleration and gyroscope values. Separately, `DW1000Ranging.loop()` is called continuously; this function handles all UWB communication in the background and invokes callbacks when data is ready.

When the DW1000 measures a distance to an anchor, the `newRange()` callback is triggered. The code identifies which anchor produced the reading by its short address and assigns it to one of four anchor slots. Invalid distances such as zero, negative, or beyond 15 m are discarded. Valid distances are stored along with a timestamp showing when that anchor was last updated.

After storing a new range, the code calls `tryTransmit()`. This function checks whether transmission conditions are met. It requires that all four anchor slots have received at least one reading at some point. It also checks that at least one of the anchor readings is recent (not older than 120 ms), ensuring the data is not completely stale. Additionally, a rate limiter ensures packets are sent no faster than once every 30 ms.

If all conditions are satisfied, the code builds a CSV string containing ten values: four UWB distances, three accelerometer values (ax, ay, az), and three gyroscope values (gx, gy, gz). This string is then sent to the base station using `esp_now_send()`. The send is non-blocking; the ESP32 immediately resumes collecting sensor data.

Anchor discovery and loss events are handled by simple callback functions that print status messages to the serial monitor, but they do not affect data transmission.

3.4.2 Anchor Code

In the `setup()` function the SPI bus is initialised using explicitly defined pins. After SPI setup, the `DW1000Ranging` library initializes communication with the DW1000 by specifying the reset, chip select, and interrupt pins. `DW1000.useSmartPower(true)` is set, allowing the chip to automatically adjust transmission power based on link conditions. This improves energy efficiency and range reliability. The antenna delay is then set to 16580 this value compensates for fixed signal delays caused by the antenna and RF path and is critical for accurate distance measurements. This value was selected after testing for various delay over increasing distances.

The UWB module is set to anchor mode using `startAsAnchor()`. The anchor is assigned:

- A unique 64-bit address
- A ranging mode (`MODE_LONGDATA_RANGE_LOWPOWER`)

Once started, the anchor immediately begins listening for UWB packets from the tag and responds as required by the two-way ranging protocol. A confirmation message is printed to the serial monitor indicating that the anchor is running.

In the `loop()` function only one call occurs – “`DW100Ranging.loop();`”. This function listens for incoming UWB messages from a tag. It handles timestamping and protocol timings and sends the response messages back to the tag.

All ranging logic is handled internally by the DW1000 library, so the Arduino code does not need to manage timing, packet construction or state machines.

3.4.3 Receiver Code

This code programmes an ESP32 into a receiver for ESP-Now messages from the ball’s ESP32 module. Its role is to listen for incoming wireless data packets, extract the contents and print them to serial monitor.

The `onDataRecv(...)` function is automatically called by the ESP-Now system whenever a packet is received. The code copies the received bytes into a local character buffer, appends a null terminator (`'\0'`), and prints the result directly to the serial port. Because the sender transmits CSV text, the receiver simply prints the CSV string exactly as received.

In the `setup()` function Serial communication is initialized for debugging and data output. The ESP32 is set to Wi-Fi Station mode and disconnected from any access point. The Wi-Fi channel is forced to channel 1, which must match the sender’s channel for ESP-NOW to work. The transmit power is set to maximum (even though this device mainly receives). ESP-NOW is initialized. The ESP-NOW PHY data rate is configured to match the transmitter’s rate. The receive callback `onDataRecv(..)` is registered so incoming packets trigger the function automatically.

The `loop()` is empty as all operation is event driven. The ESP-Now subsystem listens in the background. When a packet arrives, `onDataRecv(...)` is called automatically. No polling or timing logic is required in `loop()`.

3.4.4 Python Code

The receiver’s ESP32 produces raw sensor data to be processed in Python software. This PC-side application includes 8 files:

- *Main.py* – Application entry point, handling serial data ingestion and the processing-to-visualization pipeline.
- *Config.py* – Stores global constants, including anchor coordinates, serial port settings and filtering thresholds.
- *Kick_detector.py* – Monitors accelerometer magnitude to identify high-G impacts for kick counter.
- *Trilaterate.py* – Calculates 4 raw UWB ranges into a 3D coordinate.
- *Position_filter.py* - Smooths jittery 3D data using an Exponential Moving Average (EMA) and calculates real-time velocity.
- *Visualiser.py* - 2D dashboard, including the speedometer, IMU orientation cube, and live telemetry panels.
- *Visualiser_3d.py* - Handles OpenGL rendering for the 3D trajectory trail, ball sphere, and anchor layout.
- *Simulate.py* - Generates artificial values to test the software logic and UI without needing the physical hardware powered on.

Main.py

This Python program is the main application that processes data coming from the ESP32-based UWB tag. It runs on a computer and continuously receives distance and IMU measurements over a serial connection, converts those measurements into a 3D position, filters the motion, detects kick events, and passes the results to a visualisation system.

When the program starts, an App object is created. During initialization, it opens a serial connection using the port and baud rate defined in the config.py file. If the serial port cannot be opened, the program exits immediately. Once connected, it initializes three processing components: a position filter for smoothing and velocity estimation, a kick detector for detecting impacts from accelerometer data, and a visualiser that controls the update loop and rendering. The visualiser is configured to call the program's process() method repeatedly.

Serial data is read using the _parse_serial() method. This function checks whether data is available and reads one line at a time. Each line is expected to be a CSV string containing exactly ten values: four UWB ranges, three accelerometer values, and three gyroscope values. If the line is missing or invalid it is ignored. Valid data is parsed into individual variables and returned for further processing.

The core logic runs inside the process() method, which represents one frame of computation. First, it attempts to read and parse a new serial line. If no valid data is available, the frame is skipped. When valid data is received, the four UWB ranges are used together with the known anchor positions from the configuration file to compute a raw 3D position using trilateration. If the trilateration fails, the frame is discarded.

Once a raw position is computed, the code performs a sanity check to reject physically unrealistic results. It calculates the distance of the position from the origin and compares it to a fixed maximum radius. If the point lies outside this boundary, it is treated as an outlier and ignored. This prevents occasional UWB errors from corrupting the tracking output.

If the position passes the bounds check, it is fed into the position filter. The filter smooths the noisy UWB measurements and estimates velocity in three dimensions. At the same time, the accelerometer data is passed into the kick detector, which analyses sudden acceleration changes to determine whether a kick event has occurred and updates a running kick count.

Finally, the processed results are returned as a single data tuple containing filtered position, velocity, raw IMU data, UWB ranges, kick status, and kick count. This output is consumed by the visualiser, which displays or records the tracking results. The program continues running in this loop until it is manually stopped.

Config.py

This file defines all the fixed configuration values used by the 3D tracking system, keeping hardware settings, geometry and tuning parameters separate from the main application logic.

It specifies the serial connection settings, including the COM port and baud rate used to receive data from the ESP32 over USB.

Next, it defines the physical positions of the four UWB anchors in 3D space using Cartesian coordinates in meters. These positions are essential for trilateration, as they describe where each anchor is placed relative to a fixed reference (the origin). The commented diagram provides a rough top-down layout to help visualise the anchor arrangement.

The range index mapping section tells the software how to associate the four distance values received from the ESP32 with the corresponding anchor positions. If anchors are physically renamed or detected in a different order, these indices can be changed without modifying the processing code.

The position smoothing parameter alpha controls how aggressively the position filter smooths UWB noise. Lower values produce smoother motion, while higher values make the system more responsive but noisier.

The kick detection settings define thresholds and timing rules used to detect sudden acceleration spikes. These values control how strong a motion must be to count as a kick, how long the system waits before detecting another kick, and how long kick indicators are shown.

The visualiser settings configure how data is displayed, including buffer sizes, update rates, axis limits, and trail length. These values affect performance and readability but do not change the underlying tracking logic.

Kick_detector.py

This Python file handles all kick detection logic. It creates a class called KickDetector which has three functions.

The first method `__init__(self)` initialises two variables; `kick_count` which represents the number of kicks that have occurred and `last_kick_t` which represents the time elapsed after the last kick took place.

Next, `update(self, ax, ay, az)` is called with the current accelerometer readings in the x, y and z axes. It computes the overall magnitude of acceleration using the formula:

$$\text{magnitude of acceleration} = \sqrt{ax^2 + ay^2 + az^2}$$

If the acceleration magnitude exceeds the predefined kick threshold and enough time has passed since the previous kick, the event is classified as a kick. The kick counter is incremented, the timestamp is updated, and the method returns True to indicate a kick was detected. If these conditions are not met the method returns False for that update – no kick has taken place.

The `reset()` method clears the kick count and timing state, allowing the detector to restart fresh if needed.

Trilaterate.py

This file implements a 3D trilateration function that computes the position of a tag using distance measurements to four known anchor points. Because four anchors are used to solve for three unknowns (x, y, z), the system is over-determined, so a least-squares solution is used to obtain the best-fit position.

The function receives the 3D coordinates of four anchors from `config.py` and the measured distances from the tag to each anchor (`r1, r2, r3, r4`). It starts by forming the mathematical equations of four spheres, where each sphere represents all possible tag positions at a given distance from an anchor.

To make the problem solvable, the equation for the first anchor is subtracted from the equations for the other three anchors. This removes the squared terms and converts the system into a linear matrix equation of the form $Ax = b$, where x is the unknown position vector.

The matrix A and vector b are constructed using the anchor positions and range values. Before solving, the code checks whether the anchor geometry is degenerate (for example, anchors lying in a plane or line), which would make the solution unreliable. If the geometry is invalid, the function returns `None`.

If the system is valid, NumPy library's least-squares solver is used to compute the best-fit position. The resulting x , y , and z coordinates are returned as floating-point values. If a numerical error occurs during solving, the function safely returns `None`.

Position_filter.py

This file implements a simple position smoothing and velocity estimation filter for 3D UWB position data. It is designed to reduce noise in raw trilaterated positions while also estimating motion speed.

When a `PositionFilter` object is created, it initializes internal state for position, velocity, and timing. Until the first valid position arrives, the filter has no stored position and waits for data.

The `update()` method is called each time a new UWB position is available. On the first call, the position is stored directly and returned with zero velocity. On subsequent calls, the time difference since the last update is calculated to determine how much time has passed between measurements. If this time interval is invalid or unusually large, it is clamped to a safe default value.

The filter applies an exponential moving average (EMA) to each position axis. The smoothing factor (`ALPHA_POS`) controls how much weight is given to the new UWB measurement versus the previous filtered position. This reduces jitter while still allowing the position to respond to motion.

Velocity is then computed by dividing the change in filtered position by the elapsed time for each axis. The method returns the smoothed position along with the estimated velocity. The `reset()` method clears all stored state, allowing the filter to restart cleanly if tracking needs to be reinitialized.

Visualiser.py

This file implements the real-time visualisation layer of the tracking system. It receives processed tracking data (position, velocity, IMU values, UWB ranges, and kick events) from the main application and displays them across multiple interactive windows using `PyQtGraph` and `OpenGL`.

When a `Visualiser` object is created, it receives a callback function (`on_update`) that supplies new tracking data each frame. A Qt application is created, and three separate windows are built: a main tracking window, a sensor-data window, and a top-down 2D view. A timer is started that calls the `_tick()` method at a fixed update rate, which drives all visual updates.

The tracking window is the primary display. It contains an embedded 3D OpenGL view showing the ball's position and motion in space, a 3D IMU orientation cube that rotates based on gyroscope data, and a speedometer widget that visually displays the current speed. A live statistics panel updates numerical values such as position, height, speed, vertical velocity, spin, UWB ranges, and total kick count.

The sensor data window plots raw time-series data. It shows accelerometer values, gyroscope values, UWB ranges to each anchor, and overall speed over time. Fixed-length buffers are used so the plots scroll smoothly while maintaining a consistent history length. Threshold lines are drawn where relevant, such as the kick detection threshold on acceleration.

The 2D top-down window provides a planar view of the ball's motion. It shows the anchor positions, the current ball position, a short trailing path of recent movement, and a velocity arrow indicating direction of travel. When a kick is detected, a temporary red flash ring appears at the ball's location to highlight the event.

The `_tick()` method is called periodically by the timer and acts as the update engine. It retrieves the latest processed data via the callback, computes derived quantities such as speed and spin, and updates every visual element accordingly. This includes refreshing plots, updating numeric labels, animating the speedometer, advancing trails, and managing kick flash timers.

The IMU orientation cube is updated by integrating gyroscope values over time to estimate roll, pitch, and yaw. These angles are applied as rotations to the cube each frame, providing a visual indication of the object's orientation and spin.

Visualiser_3d.py

This file implements the 3D OpenGL visualisation used by the tracking system. Its sole responsibility is to render the ball, anchors, motion trail, velocity, spin, and kick events in a three-dimensional scene. It does not perform tracking or filtering; it only displays data provided to it.

When a `Visualiser3D` object is created, it either attaches itself to an existing OpenGL widget or creates its own window. The camera position, background colour, and view settings are configured at startup. The constructor then builds all static and dynamic visual elements, including the ground grid, anchor markers, anchor boundary box, ball trail, ball sphere, spin indicator, velocity arrow, and kick flash ring.

The ground grid provides spatial reference and is centred based on the anchor layout. A wireframe cuboid is drawn around the anchor extents to show the valid tracking volume. Each anchor is rendered as a small cube with a vertical line down to the ground and a label, making the anchor geometry easy to understand in 3D.

The ball trail is implemented as a line strip that stores a short history of recent positions. As new positions arrive, older points fade out using a colour gradient, visually indicating direction and

motion over time. The ball itself is rendered as a small, shaded sphere that moves smoothly through space.

A spin cube is drawn around the ball to indicate rotational motion. Its rotation angle is updated every frame based on the gyroscope derived spin value, giving a visual cue for how fast the ball is rotating. The cube is reset and repositioned each update to ensure it rotates cleanly around the ball.

The velocity arrow is drawn only when the ball is moving above a minimum speed. It points in the direction of motion and scales with speed, allowing quick visual interpretation of both direction and magnitude of velocity.

Kick events are highlighted using a red flashing ring that appears briefly at the ball's position when a kick is detected. A timer controls how long the ring remains visible before fading out.

The `update()` method is called once per frame with the latest position, velocity, spin, and kick information. It updates every dynamic element in the scene: extending the trail, moving the ball, rotating the spin cube, updating the velocity arrow, and handling kick flashes. All animations are time-based using the global update interval.

Simulate.py

This script provides a software-only simulation of the tracking system so the visualisation and processing pipeline can be tested without any physical UWB or IMU hardware. It generates fake motion, fake sensor data, and fake UWB ranges that closely resemble real measurements

At startup, the script computes the centre point of the anchor layout using the anchor positions defined in `config.py`. This centre is used as the reference around which the simulated ball motion occurs. The script also defines a helper function that calculates synthetic UWB ranges by measuring the true distance from a simulated position to each anchor and adding small random noise to mimic real UWB measurement error.

The `SimApp` class mirrors the structure of the real application. It creates a position filter, a kick detector, and a visualiser. Instead of reading serial data, the visualiser repeatedly calls the `process()` method to generate one simulated frame of data at a time.

Inside `process()`, a global time variable is advanced to drive motion. The simulated ball follows a smooth circular path in the horizontal plane while oscillating vertically, producing realistic 3D movement. Using this simulated position, noisy UWB ranges are generated for all four anchors. Fake IMU data is also generated. Acceleration values include gravity and random noise, while gyroscope values scale with movement speed to simulate spin. Random kick events are injected at a low probability to test kick detection and visual feedback.

The simulated UWB ranges are passed through the same trilateration function used in the real system, producing a reconstructed position. This position is then smoothed by the position filter, and velocities are computed. Kick counts are updated when simulated kicks occur.

Finally, the processed data is returned in the same format as the real tracking system, allowing the visualiser to display motion, speed, spin, trails, and kick flashes exactly as if live hardware were connected.

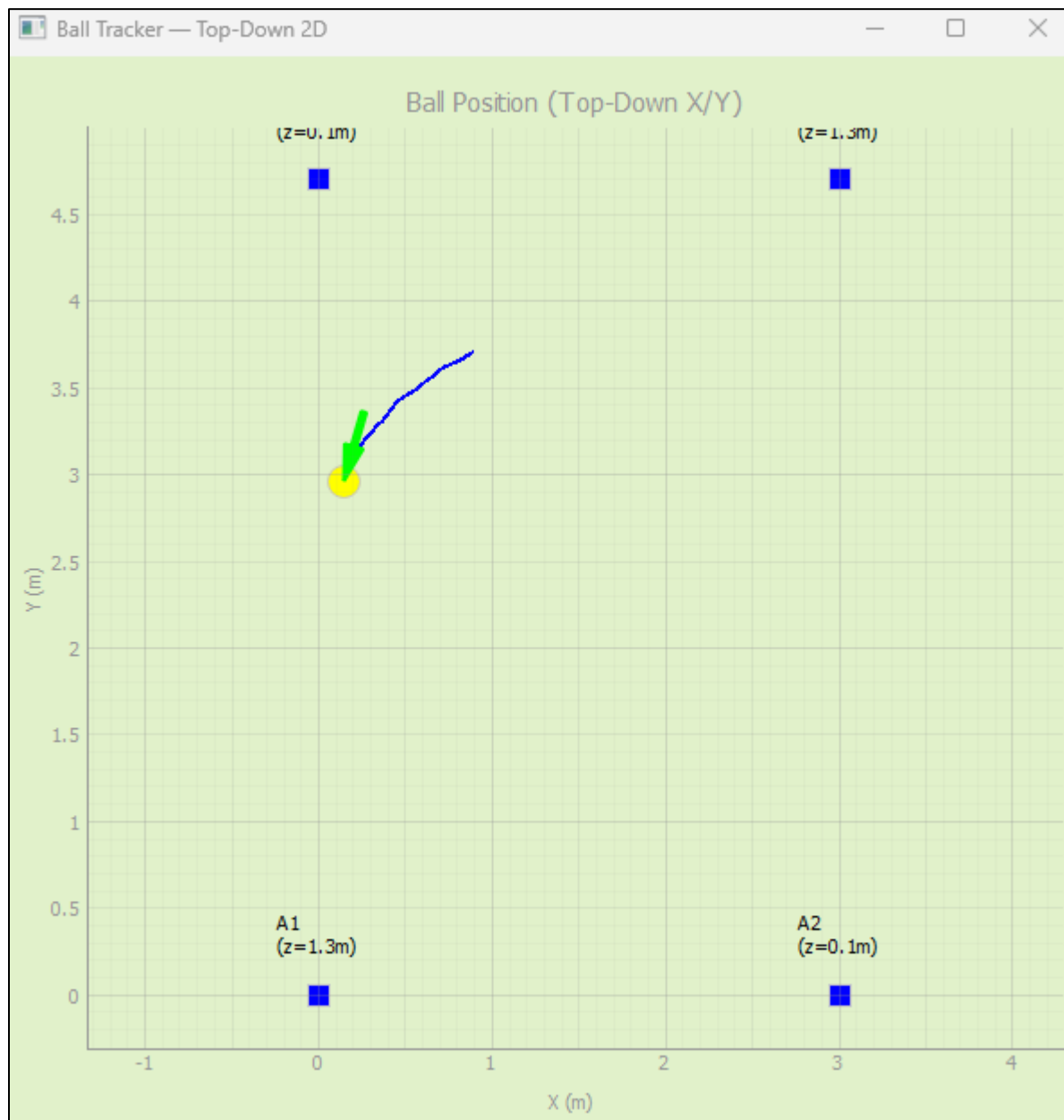


Figure 3.12: Top-Down 2D Tracking of Ball

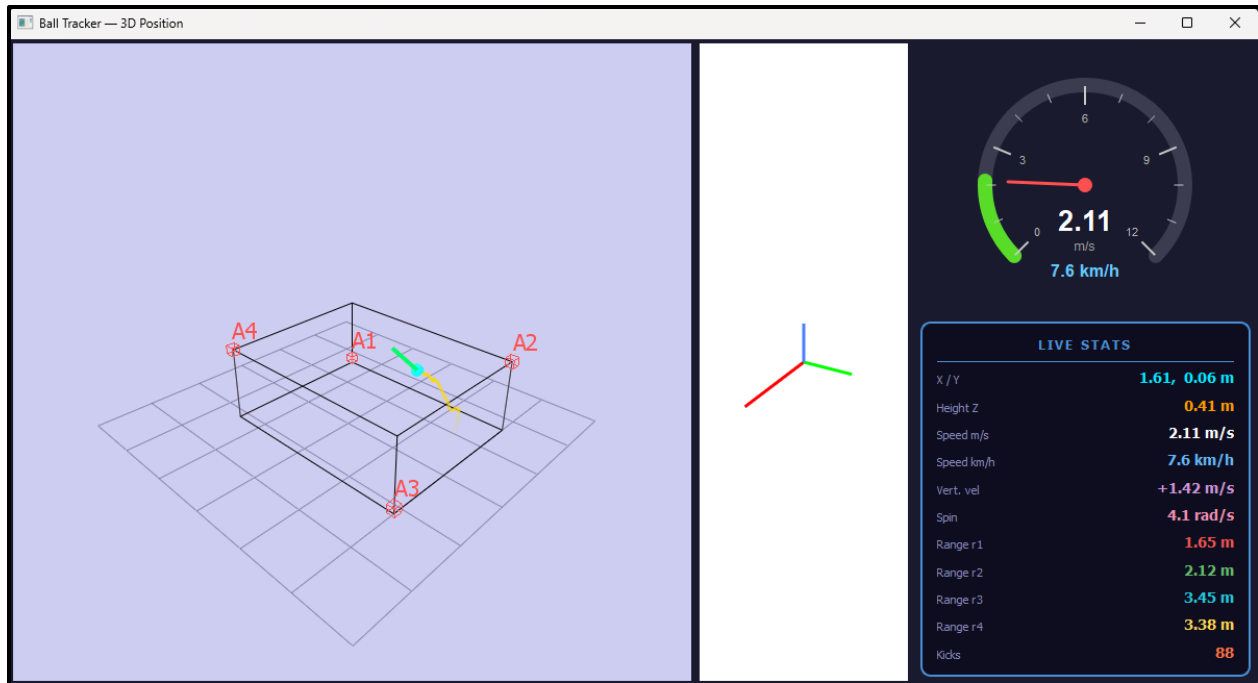


Figure 3.13: 3D Tracking Window with Speedometer & Stats Bar

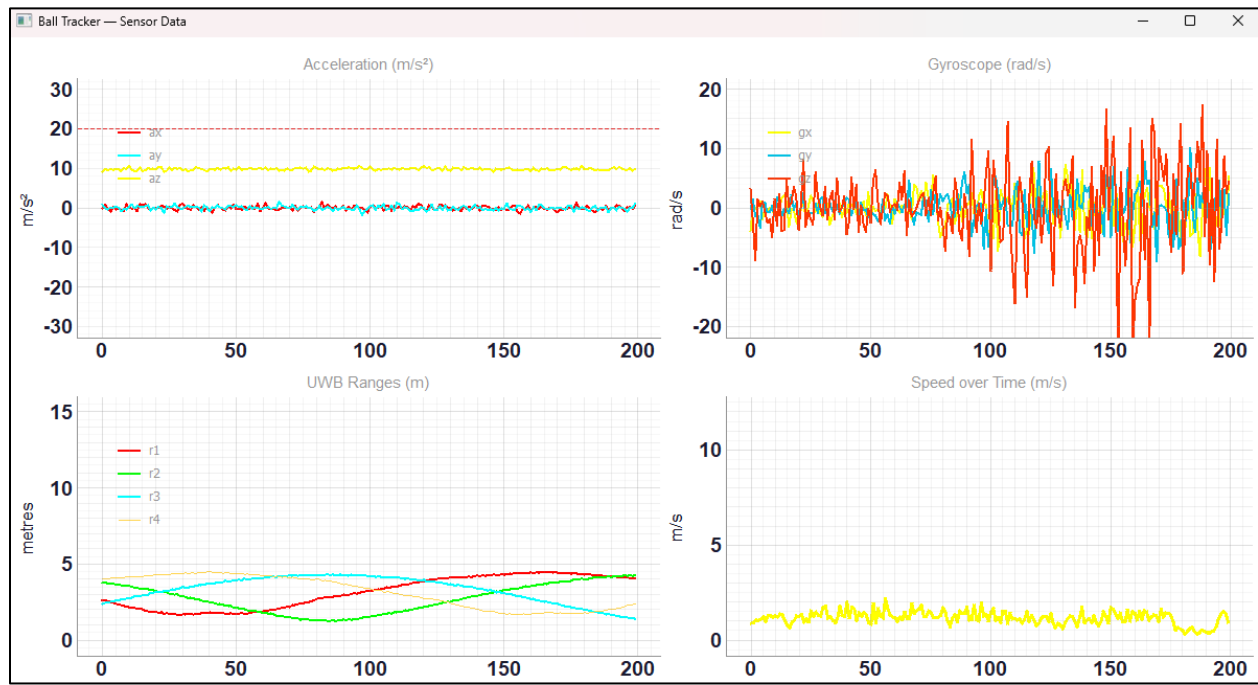


Figure 3.14: Raw Sensor Data Graphs

4 Analysis & Results

4.1 Two-Way Ranging Calibration Experiment

Before fully deploying the system, a calibration test was carried out to evaluate how accurate the UWB distance measurements were. This involved taking measurements at ten known distances and comparing them to the true values. The experiment was designed to isolate the effect of two independent variables on the ranging performance: the hardware implementation (breadboard versus a custom soldered board) and the use of antenna delay calibration. This produced four tests in total: breadboard with calibration, breadboard without calibration, custom board with calibration, and custom board without calibration. This allowed a direct assessment of each factor's contribution to the overall measurement accuracy.



Figure 4.1: Image of Calibration Testing

Antenna delay calibration corrects for the fixed timing offset introduced by the physical antenna on each DW1000 module. Without this correction, the time-of-flight measurement includes a systematic bias that leads to a ranging error. The calibration value used in the final system was an antenna delay of 16620 which was determined using a reference distance of around 3.7 m. This distance was chosen as it was the geometric centre of our playing area. Applying this calibration consistently reduced the ranging error across all distances, showing that the initial offset is a major source of inaccuracy and can be effectively corrected.

A major improvement in ranging performance also came from moving from a breadboard setup to a custom soldered board. While breadboards are useful for quick prototyping, they are not well suited for high-frequency applications like UWB. The jumper wires and loose connections can introduce noise and interference, and the lack of a proper ground plane led to unstable signal behaviour.

This was reflected in the results, where the breadboard setup showed a higher standard deviation in the distance measurements, meaning the readings were more spread out and less consistent. This led to a noisier and less reliable output.

In contrast, the custom soldered board provided short, stable connections and a proper grounding structure, which significantly reduced noise. As a result, the measurements became much more consistent, with a smoother and more predictable error behaviour. This clearly showed the benefit of moving to a more permanent and robust hardware design for the final implementation.

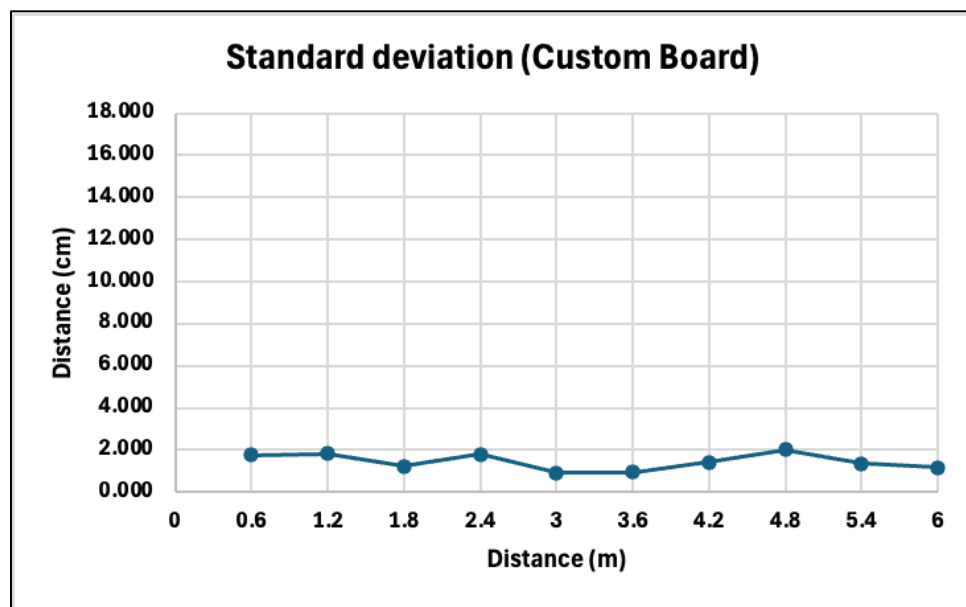


Figure 4.2: Excel Graph of Standard Deviation on Custom Board

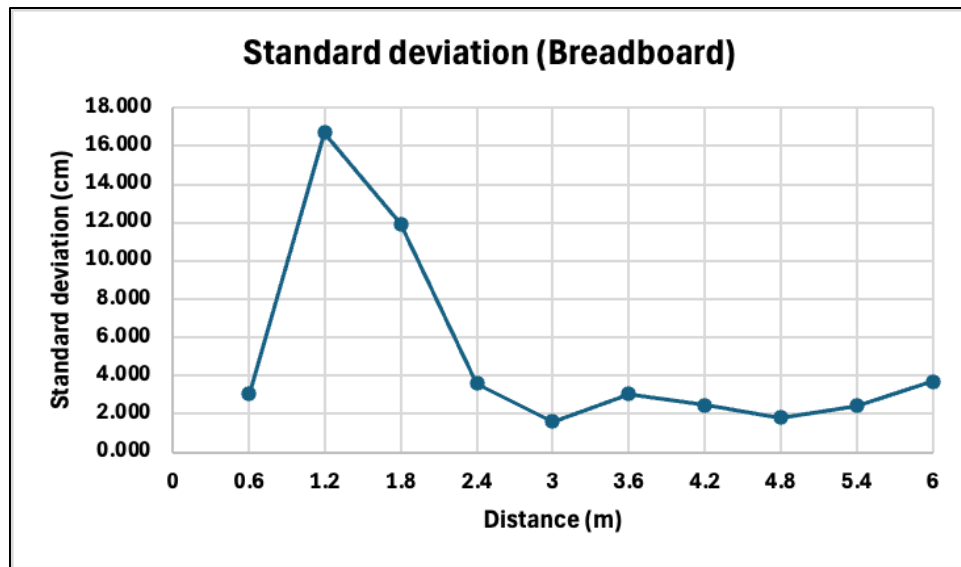


Figure 4.3: Excel Graph of Standard Deviation on Breadboard

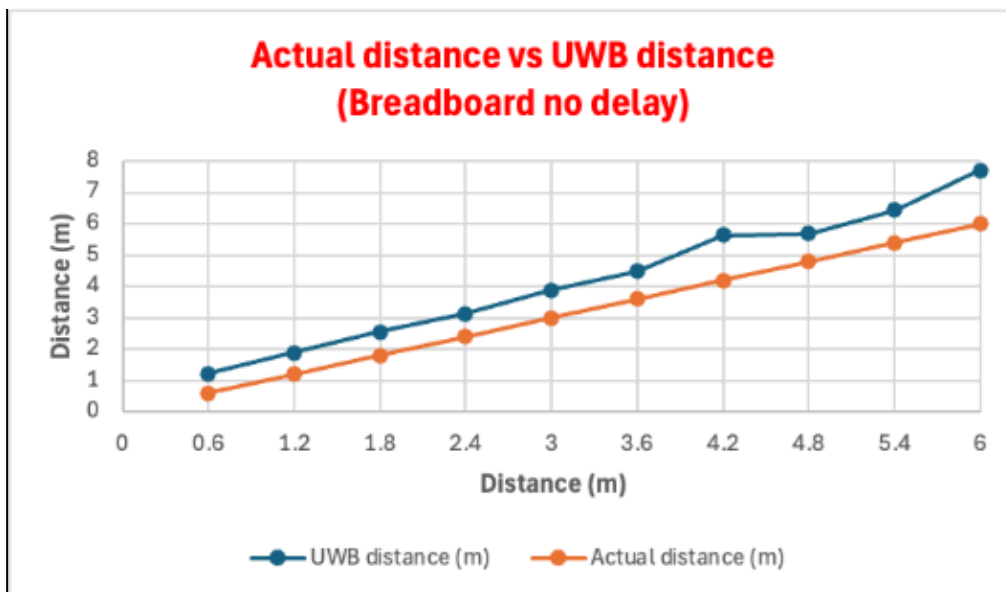


Figure 4.4: Excel Graph of Actual vs UWB distance (Breadboard with no delay)

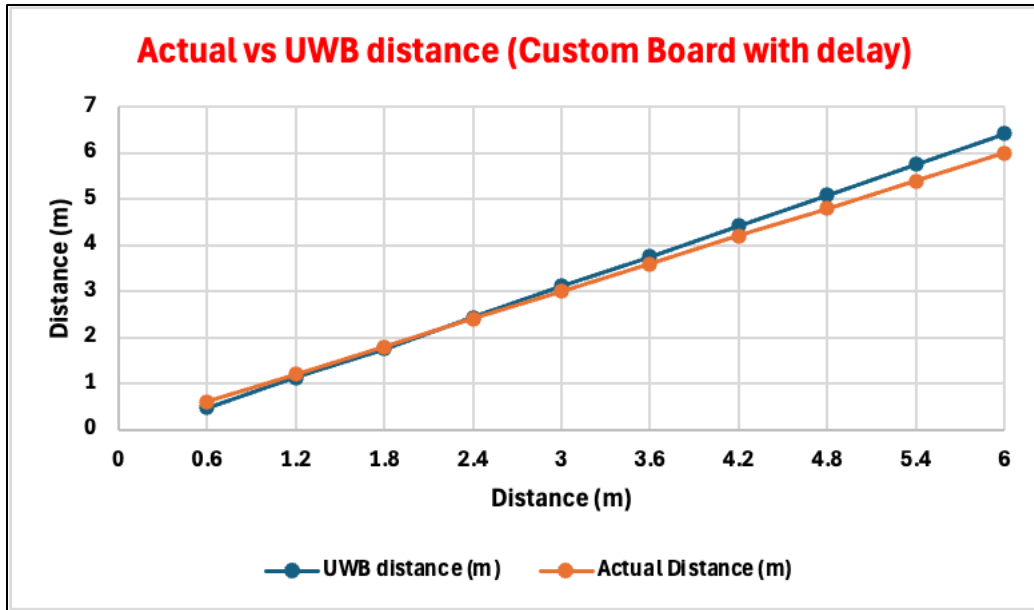


Figure 4.5: Excel Graph of Actual vs UWB distance (Custom Board with delay)
When calibrated for 2.4m

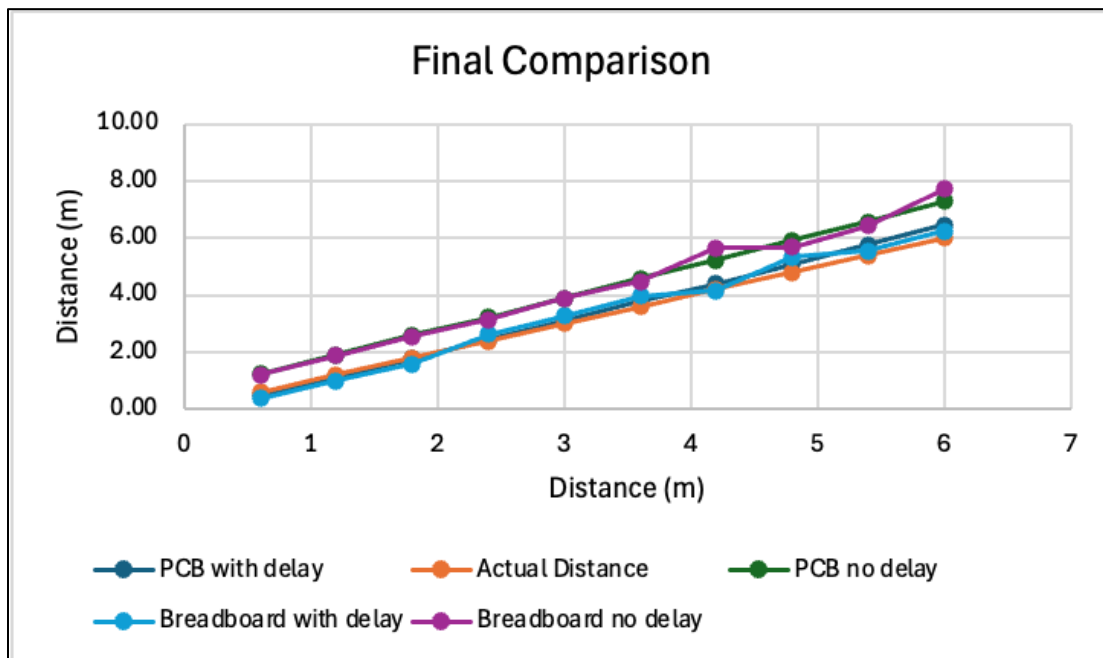


Figure 4.6: Excel Graph of Final Comparison of Multiple Hardware Tests

As shown in the graph, the measurements without antenna delay (green and purple) show a significant offset from the true distance, whereas the calibrated results closely track the actual values.

5 Conclusion

5.1 Expected Outcomes

The Smart Soccer Ball System was designed with the goal of achieving real-time, centimetre-level tracking of a soccer ball using Ultra-Wideband (UWB) technology embedded inside the ball without compromising the practical functionality of using it. It was hoped to provide accurate distance measurements to multiple anchors and use these to determine the ball's position in both 2D and 3D space.

In addition to positional tracking, the system aimed to integrate inertial sensing through an IMU to enable features such as kick detection, ball speed estimation, and spin analysis. A way to visualise all the data was also expected, where the sensor data would be wirelessly transmitted to a host laptop, processed in real time, and visualised through a user interface.

Overall, the project aimed to demonstrate a compact, reliable, and real-time tracking system suitable for sports analysis applications.

5.2 Project Achievements

The main goal of the Smart Soccer Ball System was successfully achieved. The final system was capable of tracking the ball's position in real time with centimetre-level accuracy using embedded UWB technology. Results from the experiments showed an average ranging error of 4.2 cm, with a maximum error of 9.1 cm over distances of up to 6 metres, meeting the accuracy targets set at the beginning of the project.

Beyond just positional accuracy, the system successfully brings together a sensing and communication system in a compact embedded setup. The ball sends four UWB distance measurements along with the IMU data (three accelerometer and three gyroscope) to the laptop at around 16Hz using the ESP-NOW and USB serial bridge, where the data is filtered, processed, and rendered in real time. Features such as kick detection, ball speed estimation, spin visualisation, live 2D and 3D position tracking are all working and shown together in the Python interface. The transition from a 2D tracking system to a full 3D system using four anchors in a rectangular cuboid configuration was also achieved, with the anchor placement strategy successfully minimising geometric dilution of precision across all the three spatial axes. The Exponential Moving Average (EMA) filtering applied to the incoming range measurements provides adequate smoothing to produce stable position estimates without introducing noticeable lag into the live display.

Although the system was tested in a controlled indoor environment, the results show that UWB-based smart ball tracking is a practical and effective approach for sports analysis. The overall system design, combining embedded sensors, wireless communication, and a Python-based interface, provides a strong foundation for future improvements such as PCB implementation, wireless charging, and outdoor use.



Figure 5.1: Complete System Testing

5.3 Challenges Faced

Several technical and practical challenges were encountered throughout the project, many of which required changes to the original design approach.

One of the first major challenges was physically embedding the electronics inside the ball. The initial plan was to place the hardware inside a standard soccer ball by cutting open a panel and inserting a compact electronics module. In practice, it was very difficult to make the hardware small enough to fit without distorting the ball, and re-sealing it properly after installation was impractical. This led to a change in design, where an indoor foam ball was used instead. The foam ball could be easily cut in half, fitted with a 3D-printed housing at its centre, and reconstructed using Velcro straps. While this was a practical solution, it moved away from the original plan of using a regular ball and keeping the aerodynamics properties held by it.

Another major challenge was signal quality in the UWB hardware. Early testing using a breadboard prototype produced very unstable and inconsistent distance measurements. This was linked to the poor high-frequency performance of breadboards, including noise, interference, and lack of proper grounding. The loose connections with the jumper wires also likely played a significant role in poor performance. Moving to a custom soldered board resolved these issues and resulted in much more stable and predictable measurements, which were paramount for accurate calibration.

Calibrating the antenna delay was also more complex and tedious than expected. The DW1000 brings a fixed timing offset that causes a consistent error in distance measurements if not corrected. Finding the correct calibration value required repeated testing at a known distance and adjusting the delay until the error was minimised. This process had to be repeated whenever the hardware setup (i.e. dimensions of the playing area) changed.

Coordinating the communication between four anchors and the ball also required careful management. Since ESP-NOW has no built-in collision avoidance, packets are lost if multiple anchors transmit at the same time. To avoid this, the ranging sequence was structured so that each anchor communicated with the tag one at a time rather than simultaneously. This added further complexity to setting up the system.

An increase in the sampling rate for the UWB readings was another challenge. The aim was to achieve the range of 20-30Hz by adjusting the timing and reply times within the DW1000 library's source code, however any change would result in instability so the default values were utilised leading to the achieved sampling range of 15-20Hz.

Finally, multipath interference remained the main source of error in the final system. In indoor environments, signals reflect off walls and other surfaces, which means the receiver can pick up delayed versions of the signal as well as the direct path. This leads to slightly overestimated distances and introduces errors that cannot be completely removed with calibration, making it the main culprit for inconsistent or incorrect readings.

5.4 Recommendation for Future Work

Given a broader timeline and with the knowledge gained throughout the two semesters, several improvements and additions could be added which would greatly expand upon the scope of the project.

Event detection was an idea considered very early in planning on the project but was never developed upon due to the lack of extreme precision required for such a feature. This would include detecting events such as goals being scored or number of kicks counted before the floor hits the ground.

A custom PCB could be considered designing and ordering to reduce the physical footprint of the hardware placed within a football. If this were to be achieved in the future, it would be viable to consider integrating the PCB into an actual football using techniques researched initially.

Expanding upon the PCB design, wireless charging provides a solution to the inaccessibility of the internal hardware components when certain hardware integration techniques are applied to an actual football in regard to charging an internal battery connected to the PCB.

Increasing the robustness of the system to allow use outdoors should also be developed. The current project was designed with indoor use in mind. If the internal tag hardware was safely inserted into an actual football and the anchors were designed for outdoor conditions such as rain or snow, this could be an avenue in a future iteration of this system.

6 References

1. UWB DW1000 library used: <https://github.com/thotro/arduino-dw1000>
2. Image of Adidas MiCoach: <https://www.facebook.com/adidas/photos/micoach-smart-ballfeaturing-built-in-sensors-designed-to-improve-technique-power/584870381535885/>
3. Trilateration Reading: <https://www.mdpi.com/1424-8220/23/12/5710>
4. UWB for Sports Tracking: <https://doi.org/10.3390/s18010168>
5. GPS vs UWB: <https://www.sciencedirect.com/science/article/pii/S2665917420300337>
6. Processing Arduino data in Python: <https://www.youtube.com/watch?v=fIlkIRIuXoY&list=WL&index=2&t=5s>
7. IMU product page: <https://www.mikroe.com/6dof-imu-7-click>
8. UWB DW1000 product page: <https://www.mikroe.com/uwb-click>
9. Adidas World Cup Ball inspiration: <https://news.adidas.com/football/adidas-reveals-the-first-fifa-world-cup--official-match-ball-featuring-connected-ball-technology/s/cccb7187-a67c-4166-b57d-2b28f1d36fa0>
10. Multipath Interference and ToA: https://www.researchgate.net/publication/224395249_Ranging_With_Ultrawide_Bandwidth_Signals_in_Multipath_Environments
11. ESP-NOW: <https://randomnerdtutorials.com/esp-now-esp32-arduino-ide/>
- 12.

7 Appendix

7.1 Microcontroller Source Code

7.1.1 Anchor Source Code

```
#include <SPI.h>
#include <DW1000Ranging.h>

#define PIN_RST 27
#define PIN_SS 5
#define PIN_IRQ 34

char anchorAddress[] = "82:17:5B:D5:A9:9A:E2:03";

void setup() {
    Serial.begin(115200);
    delay(1000);

    SPI.begin(18, 19, 23);
    DW1000Ranging.initCommunication(PIN_RST, PIN_SS, PIN_IRQ);

    DW1000.useSmartPower(true);
    DW1000.setAntennaDelay(16580); //16600 for short lowp, //16620 for long lowp
    FAST //16580 for range lowp

    DW1000Ranging.startAsAnchor(
        anchorAddress,
        DW1000.MODE_LONGDATA_RANGE_LOWPOWER,
        0xE203
    );

    Serial.println("UWB ANCHOR started");
}

void loop() {
    DW1000Ranging.loop();
}
```

7.1.2 Ball Source Code

```
#include <SPI.h>
#include <WiFi.h>
#include <esp_now.h>
#include <esp_wifi.h>
#include <Wire.h>
#include <Adafruit_ICM20649.h>
#include <Adafruit_Sensor.h>
#include <DW1000Ranging.h>

// ----- PIN CONFIG -----
#define PIN_RST 32
#define PIN_SS 15
#define PIN_IRQ 21
#define SDA_PIN 16
#define SCL_PIN 17
// -----

// ----- ESP-NOW -----
uint8_t baseStationMAC[] = {0xE4, 0xB0, 0x63, 0xAE, 0xED, 0x80};
// -----

// ----- UWB -----
char tagAddress[] = "7D:00:22:EA:82:60:3B:9C";
const int MAX_ANCHORS = 4;
uint16_t seenAnchors[MAX_ANCHORS] = {0};
float ranges[MAX_ANCHORS] = {0};
unsigned long lastUpdate[MAX_ANCHORS] = {0};

const unsigned long STALE_MS = 120;
const unsigned long SEND_INTERVAL_MS = 30; // ~25Hz
unsigned long lastSend = 0;
// -----

// ----- IMU -----
Adafruit_ICM20649 icm;
float ax = 0, ay = 0, az = 0;
float gx = 0, gy = 0, gz = 0;
unsigned long lastIMU = 0;
// -----
```

```

void newRange();
void newDevice(DW1000Device* d);
void inactiveDevice(DW1000Device* d);
void tryTransmit();

void onSent(const wifi_tx_info_t *tx_info, esp_now_send_status_t status) {}

void setup() {
    Serial.begin(115200);
    delay(1000);

    // ----- ESP-NOW -----
    WiFi.mode(WIFI_STA);
    WiFi.disconnect();
    esp_wifi_set_channel(1, WIFI_SECOND_CHAN_NONE);
    esp_wifi_set_max_tx_power(84);

    if (esp_now_init() != ESP_OK) {
        Serial.println("ESP-NOW init failed");
        return;
    }

    esp_wifi_config_espnow_rate(WIFI_IF_STA, WIFI_PHY_RATE_9M);
    esp_now_register_send_cb(onSent);

    esp_now_peer_info_t peer = {};
    memcpy(peer.peer_addr, baseStationMAC, 6);
    peer.channel = 1;
    peer.encrypt = false;
    if (esp_now_add_peer(&peer) != ESP_OK) {
        Serial.println("Failed to add peer");
        return;
    }

    // ----- IMU -----
    Wire.begin(SDA_PIN, SCL_PIN);
    Wire.setClock(400000);
    if (!icm.begin_I2C()) {
        Serial.println("IMU init failed!");
    } else {
        Serial.println("IMU Ready");
    }

    // ----- UWB -----
    SPI.begin();

```

```

DW1000Ranging.initCommunication(PIN_RST, PIN_SS, PIN_IRQ);

DW1000.useSmartPower(true);
DW1000Ranging.attachNewRange(newRange);
DW1000Ranging.attachNewDevice(newDevice);
DW1000Ranging.attachInactiveDevice(inactiveDevice);

//DW1000Ranging.useRangeFilter(true); //use based on testing

DW1000Ranging.startAsTag(tagAddress, DW1000.MODE_LONGDATA_RANGE_LOWPOWER);

Serial.println("System Online");
}

void loop() {
    if (millis() - lastIMU >= 10) {
        sensors_event_t accel_evt, gyro_evt, temp_evt;
        icm.getEvent(&accel_evt, &gyro_evt, &temp_evt);
        ax = accel_evt.acceleration.x;
        ay = accel_evt.acceleration.y;
        az = accel_evt.acceleration.z;
        gx = gyro_evt.gyro.x;
        gy = gyro_evt.gyro.y;
        gz = gyro_evt.gyro.z;
        lastIMU = millis();
    }

    DW1000Ranging.loop();
}

void newRange() {
    DW1000Device* dev = DW1000Ranging.getDistantDevice();
    if (!dev) return;

    float dist = dev->getRange();
    uint16_t addr = dev->getShortAddress();

    if (dist <= 0.0 || dist > 15.0) return;

    int index = -1;
    for (int i = 0; i < MAX_ANCHORS; i++) {
        if (seenAnchors[i] == addr) { index = i; break; }
    }
    if (index == -1) {
        for (int i = 0; i < MAX_ANCHORS; i++) {

```

```

        if (seenAnchors[i] == 0) { seenAnchors[i] = addr; index = i; break; }
    }
}
if (index == -1) return;

ranges[index]      = dist;
lastUpdate[index] = millis();

tryTransmit();
}

void tryTransmit() {
    unsigned long now = millis();

    // All 4 anchors must have at least one reading
    bool allHaveData = lastUpdate[0] && lastUpdate[1] &&
                      lastUpdate[2] && lastUpdate[3];
    if (!allHaveData) return;

    // At least one must be fresh
    bool anyFresh = ((now - lastUpdate[0]) < STALE_MS) ||
                   ((now - lastUpdate[1]) < STALE_MS) ||
                   ((now - lastUpdate[2]) < STALE_MS) ||
                   ((now - lastUpdate[3]) < STALE_MS);
    if (!anyFresh) return;

    if ((now - lastSend) < SEND_INTERVAL_MS) return;
    lastSend = now;

    // CSV: r1,r2,r3,r4,ax,ay,az,gx,gy,gz (10 values)
    char csv[160];
    snprintf(csv, sizeof(csv),
        "%.3f,%.3f,%.3f,%.3f,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f",
        ranges[0], ranges[1], ranges[2], ranges[3],
        ax, ay, az,
        gx, gy, gz
    );
    esp_now_send(baseStationMAC, (uint8_t*)csv, strlen(csv));
}

void newDevice(DW1000Device* dev)      { Serial.println("Anchor Found"); }
void inactiveDevice(DW1000Device* dev) { Serial.print("Anchor Lost");   }

```

7.1.3 Receiver Source Code

```
#include <WiFi.h>
#include <esp_now.h>
#include <esp_wifi.h>    // needed for esp_wifi_set_channel, tx power, data rate

void onDataRecv(const uint8_t *mac, const uint8_t *incomingData, int len) {
    char buffer[150];
    memcpy(buffer, incomingData, len);
    buffer[len] = '\0';

    // Print ONLY the CSV
    Serial.println(buffer);
}

void setup() {
    Serial.begin(115200);
    delay(2000);

    WiFi.mode(WIFI_STA);
    WiFi.disconnect();

    // Lock to channel 1 – must match tag and anchors
    esp_wifi_set_channel(1, WIFI_SECOND_CHAN_NONE);

    // Maximum TX power
    esp_wifi_set_max_tx_power(84);

    if (esp_now_init() != ESP_OK) {
        Serial.println("ESP-NOW init failed");
        return;
    }

    // 2 Mbps PHY rate – must match the tag
    esp_wifi_config_espnow_rate(WIFI_IF_STA, WIFI_PHY_RATE_9M);

    esp_now_register_recv_cb(onDataRecv);

    Serial.println("Receiver ready");
}
```

```
void loop() {}
```

7.2 Python Monitor Source Code

7.2.1 Main.py

```
# =====
# main.py — 3D tracking with 4 anchors
# =====

import sys
import time
import serial
import math

import config
from trilaterate import trilaterate
from position_filter import PositionFilter
from kick_detector import KickDetector
from visualiser import Visualiser

class App:
    def __init__(self):
        try:
            self.ser = serial.Serial(config.PORT, config.BAUD, timeout=0.1)
            time.sleep(2)
            print(f"Connected to {config.PORT}")
        except Exception as e:
            print(f"Failed to open serial port {config.PORT}: {e}")
            sys.exit(1)

        self.pos_filter = PositionFilter()
        self.kick_det = KickDetector()
        self.vis = Visualiser(on_update=self.process)

    def _parse_serial(self):
        """
        Read one CSV line from serial.
        Expected format (10 values):
        r1,r2,r3,r4,ax,ay,az,gx,gy,gz
        Returns (r1,r2,r3,r4,ax,ay,az,gx,gy,gz) or None.
        """
        if not self.ser.in_waiting:
            return None
```

```

line = self.ser.readline().decode('utf-8', errors='ignore').strip()
if not line:
    return None

parts = line.split(",")
if len(parts) != 10:
    return None

values = [float(p) for p in parts]

r1 = values[config.IDX_ANCHOR_1]
r2 = values[config.IDX_ANCHOR_2]
r3 = values[config.IDX_ANCHOR_3]
r4 = values[config.IDX_ANCHOR_4]
ax, ay, az = values[4], values[5], values[6]
gx, gy, gz = values[7], values[8], values[9]

return r1, r2, r3, r4, ax, ay, az, gx, gy, gz

def process(self):
    try:
        parsed = self._parse_serial()
        if parsed is None:
            return None

        r1, r2, r3, r4, ax, ay, az, gx, gy, gz = parsed

        # 1. Calculate raw position via trilateration
        pos = trilaterate(config.A1, config.A2, config.A3, config.A4,
                          r1, r2, r3, r4)
        if pos is None:
            return None

        uwb_x, uwb_y, uwb_z = pos

        # ---- 2. THRESHOLD CHECK (ADD THIS) ----
        # Define your limit (e.g., 10.0 meters from the center)
        MAX_DISTANCE = 7.0

        # Calculate distance from origin (0,0,0)
        distance = math.sqrt(uwb_x**2 + uwb_y**2 + uwb_z**2)

        # If the calculated point is too far away, ignore this frame
        if distance > MAX_DISTANCE:
            # Optional: print(f"Outlier ignored: {distance:.2f}m")

```

```
        return None

    # -----

    # 3. If sane, pass to filters
    x, y, z, vx, vy, vz = self.pos_filter.update(uwb_x, uwb_y, uwb_z)
    kick = self.kick_det.update(ax, ay, az)

    return (x, y, z, vx, vy, vz,
            ax, ay, az,
            gx, gy, gz,
            r1, r2, r3, r4,
            kick, self.kick_det.kick_count)

except (ValueError, UnicodeDecodeError):
    return None
except Exception as e:
    print(f"Error: {e}")
    return None

def run(self):
    self.vis.run()

if __name__ == "__main__":
    App().run()
```

7.2.2 Config.py

```
# =====
# config.py — 3D tracking with 4 anchors
# =====

# ----- SERIAL -----
PORT = "COM10"
BAUD = 115200

# ----- ANCHOR POSITIONS (x, y, z) in meters -----
#
# Top-down layout example:
#
# A2(0, 2.4, 0) ----- A4(2.4, 2.4, 2.0) <- elevated
#   |                   |
#   |                   |
# A1(0, 0, 0) ----- A3(2.4, 0, 2.0) <- elevated
#
A1 = (0.0, 0.0, 1.30)
A2 = (3.0, 0.0, 0.1)
A3 = (0.0, 4.7, 0.1)
A4 = (3.0, 4.7, 1.3)

# ----- RANGE INDEX MAPPING -----
# Maps anchor slot 0-3 from the ESP32 CSV to A1-A4 above.
# If position looks wrong, swap these around.
IDX_ANCHOR_1 = 0
IDX_ANCHOR_2 = 1
IDX_ANCHOR_3 = 2
IDX_ANCHOR_4 = 3

# ----- POSITION SMOOTHING -----
ALPHA_POS = 0.2 # lower = smoother, higher = more responsive

# ----- KICK DETECTION -----
KICK_THRESHOLD = 20.0
KICK_COOLDOWN_S = 1.0
KICK_FLASH_MS = 500

# ----- VISUALISER -----
TRAIL_LENGTH = 30
BUFFER_LEN = 200
UPDATE_MS = 20
```

```
SPEED_Y_MAX = 12
ACCEL_Y_RANGE = 30
RANGE_Y_MAX = 15.0
HEIGHT_Y_MAX = 5.0 # max height shown on Z plot
```

7.2.3 Kick_detector.py

```
# =====
# kick_detector.py
# =====

import math
import time
from config import KICK_THRESHOLD, KICK_COOLDOWN_S

class KickDetector:
    def __init__(self):
        self.kick_count = 0
        self.last_kick_t = 0.0

    def update(self, ax, ay, az):
        magnitude = math.sqrt(ax**2 + ay**2 + az**2)
        now = time.time()

        if magnitude > KICK_THRESHOLD and now - self.last_kick_t > KICK_COOLDOWN_S:
            self.kick_count += 1
            self.last_kick_t = now
            return True

        return False

    def reset(self):
        self.kick_count = 0
        self.last_kick_t = 0.0
```

7.2.4 Trilaterate.py

```
# =====
# trilaterate.py — 3D trilateration with 4 anchors
# =====
#
# With 4 anchors in 3D we have an over-determined system
# (4 equations, 3 unknowns). We use least-squares via
# numpy to get the best-fit solution rather than picking
# 3 anchors and discarding the 4th.
#
# Each sphere equation:
#  $(x-x_i)^2 + (y-y_i)^2 + (z-z_i)^2 = r_i^2$ 
#
# Subtracting anchor 1's equation from anchors 2, 3, 4
# linearises the system into  $Ax = b$  form.
# =====

import numpy as np

def trilaterate(p1, p2, p3, p4, r1, r2, r3, r4):
    """
    Solve 3D position from 4 anchor positions and ranges.

    Args:
        p1..p4 : (x, y, z) tuples for each anchor
        r1..r4 : measured ranges to each anchor (metres)

    Returns:
        (x, y, z) or None if system is degenerate
    """
    anchors = [p1, p2, p3, p4]
    ranges = [r1, r2, r3, r4]

    x1, y1, z1 = p1

    # Build linearised system by subtracting anchor 1's equation
    # from each of the remaining anchors
    A_rows = []
    b_rows = []

    for i in range(1, 4):
        xi, yi, zi = anchors[i]
        ri = ranges[i]

        A_rows.append([
```

```

        2 * (xi - x1),
        2 * (yi - y1),
        2 * (zi - z1)
    ])
    b_rows.append(
        r1**2 - ri**2
        - x1**2 + xi**2
        - y1**2 + yi**2
        - z1**2 + zi**2
    )

A = np.array(A_rows, dtype=float)
b = np.array(b_rows, dtype=float)

# Check for degenerate geometry (anchors coplanar/colinear)
if abs(np.linalg.det(A)) < 1e-6:
    return None

try:
    pos, _, _ = np.linalg.lstsq(A, b, rcond=None)
    return float(pos[0]), float(pos[1]), float(pos[2])
except np.linalg.LinAlgError:
    return None

```

7.2.5 Position_filter.py

```
# =====
# position_filter.py — EMA smoothing on 3D UWB position
# =====

import time
from config import ALPHA_POS

class PositionFilter:
    """
    EMA filter on trilaterated 3D position.
    Velocity back-computed from position change over dt.
    """

    def __init__(self):
        self.x = None
        self.y = None
        self.z = None
        self.vx = 0.0
        self.vy = 0.0
        self.vz = 0.0
        self.last_time = None

    def update(self, uwb_x, uwb_y, uwb_z):
        now = time.time()

        if self.x is None:
            self.x = uwb_x
            self.y = uwb_y
            self.z = uwb_z
            self.last_time = now
            return self.x, self.y, self.z, self.vx, self.vy, self.vz

        dt = now - self.last_time
        self.last_time = now

        if dt <= 0 or dt > 0.5:
            dt = 0.05

        prev_x, prev_y, prev_z = self.x, self.y, self.z

        self.x = ALPHA_POS * uwb_x + (1 - ALPHA_POS) * self.x
        self.y = ALPHA_POS * uwb_y + (1 - ALPHA_POS) * self.y
        self.z = ALPHA_POS * uwb_z + (1 - ALPHA_POS) * self.z
```

```
self.vx = (self.x - prev_x) / dt  
self.vy = (self.y - prev_y) / dt  
self.vz = (self.z - prev_z) / dt
```

```
return self.x, self.y, self.z, self.vx, self.vy, self.vz
```

```
def reset(self):
```

```
    self.x = None
```

```
    self.y = None
```

```
    self.z = None
```

```
    self.vx = 0.0
```

```
    self.vy = 0.0
```

```
    self.vz = 0.0
```

```
    self.last_time = None
```

7.2.6 Visualiser.py

```
# =====
# visualiser.py — 3D tracking, 4 anchors
# =====
#
# Window 1 — Tracking:
# Left      : 2D top-down position plot (X/Y)
# Centre-top : Ball height over time (Z)
# Right     : Arc speedometer + LIVE STATS panel
#
# Window 2 — Sensor Data:
# Top left  : Raw acceleration (ax, ay, az)
# Top right : UWB ranges (r1, r2, r3, r4)
# Bottom    : Speed over time
# =====

import math
import numpy as np
from collections import deque

import pyqtgraph as pg
import pyqtgraph.opengl as gl
from pyqtgraph.Qt import QtCore, QtWidgets, QtGui

from config import (
    A1, A2, A3, A4,
    TRAIL_LENGTH, BUFFER_LEN, UPDATE_MS,
    ACCEL_Y_RANGE, SPEED_Y_MAX, RANGE_Y_MAX,
    KICK_FLASH_MS, KICK_THRESHOLD
)
from visualiser_3d import Visualiser3D

# =====
# Speedometer widget
# =====

class Speedometer(QtWidgets.QWidget):
    MAX_SPEED_MS = 12.0

    def __init__(self, parent=None):
        super().__init__(parent)
        self.speed_ms = 0.0
        self.setMinimumSize(240, 240)
        self.setStyleSheet("background-color: #1a1a2e;")
```

```

def set_speed(self, speed_ms):
    self.speed_ms = max(0.0, speed_ms)
    self.update()

def paintEvent(self, event):
    painter = QtGui.QPainter(self)
    painter.setRenderHint(QtGui.QPainter.RenderHint.Antialiasing)

    w, h = self.width(), self.height()
    cx = w // 2
    cy = int(h * 0.52)
    radius = int(min(w, h) * 0.38)

    arc_rect = QtCore.QRectF(cx-radius, cy-radius, radius*2, radius*2)

    pen = QtGui.QPen(QtGui.QColor(60, 60, 80), 14)
    pen.setCapStyle(QtCore.Qt.PenCapStyle.RoundCap)
    painter.setPen(pen)
    painter.drawArc(arc_rect, 225*16, -270*16)

    fraction = min(self.speed_ms / self.MAX_SPEED_MS, 1.0)
    if fraction < 0.5:
        r, g = int(255 * fraction * 2), 220
    else:
        r, g = 220, int(220 * (1 - (fraction - 0.5) * 2))
    pen2 = QtGui.QPen(QtGui.QColor(r, g, 40), 14)
    pen2.setCapStyle(QtCore.Qt.PenCapStyle.RoundCap)
    painter.setPen(pen2)
    painter.drawArc(arc_rect, 225*16, int(-270 * fraction * 16))

    for i in range(13):
        frac = i / 12
        angle_rad = math.radians(225 - frac * 270)
        is_major = (i % 3 == 0)
        inner_r = radius - (18 if is_major else 10)

        x1 = cx + inner_r * math.cos(angle_rad)
        y1 = cy - inner_r * math.sin(angle_rad)
        x2 = cx + (radius - 2) * math.cos(angle_rad)
        y2 = cy - (radius - 2) * math.sin(angle_rad)

        painter.setPen(QtGui.QPen(QtGui.QColor(200, 200, 200), 2 if is_major else 1))
        painter.drawLine(QtCore.QPointF(x1, y1), QtCore.QPointF(x2, y2))

```

```

if is_major:
    label_r = radius - 32
    lx = cx + label_r * math.cos(angle_rad)
    ly = cy - label_r * math.sin(angle_rad)
    painter.setFont(QtGui.QFont("Arial", 8))
    painter.setPen(QtGui.QPen(QtGui.QColor(180, 180, 180)))
    painter.drawText(
        QtCore.QRectF(lx-14, ly-8, 28, 16),
        QtCore.Qt.AlignmentFlag.AlignCenter,
        str(round(frac * self.MAX_SPEED_MS))
    )

needle_rad = math.radians(225 - fraction * 270)
nx = cx + (radius - 22) * math.cos(needle_rad)
ny = cy - (radius - 22) * math.sin(needle_rad)
painter.setPen(QtGui.QPen(QtGui.QColor(255, 80, 80), 3,
    QtCore.Qt.PenStyle.SolidLine,
    QtCore.Qt.PenCapStyle.RoundCap))
painter.drawLine(QtCore.QPointF(cx, cy), QtCore.QPointF(nx, ny))

painter.setBrush(QtGui.QBrush(QtGui.QColor(255, 80, 80)))
painter.setPen(QtCore.Qt.PenStyle.NoPen)
painter.drawEllipse(QtCore.QPointF(cx, cy), 7, 7)

painter.setFont(QtGui.QFont("Arial", 20, QtGui.QFont.Weight.Bold))
painter.setPen(QtGui.QPen(QtGui.QColor(255, 255, 255)))
painter.drawText(
    QtCore.QRectF(cx-60, cy + radius*0.18, 120, 36),
    QtCore.Qt.AlignmentFlag.AlignCenter,
    f"{self.speed_ms:.2f}"
)
painter.setFont(QtGui.QFont("Arial", 9))
painter.setPen(QtGui.QPen(QtGui.QColor(160, 160, 160)))
painter.drawText(
    QtCore.QRectF(cx-40, cy + radius*0.18 + 32, 80, 18),
    QtCore.Qt.AlignmentFlag.AlignCenter, "m/s"
)
painter.setFont(QtGui.QFont("Arial", 13, QtGui.QFont.Weight.Bold))
painter.setPen(QtGui.QPen(QtGui.QColor(100, 200, 255)))
painter.drawText(
    QtCore.QRectF(cx-60, cy + radius*0.18 + 52, 120, 26),
    QtCore.Qt.AlignmentFlag.AlignCenter,
    f"{self.speed_ms * 3.6:.1f} km/h"
)
painter.end()

```

```

# =====
# Visualiser
# =====

class Visualiser:
    def __init__(self, on_update):
        self.on_update = on_update

        self.app = QtWidgets.QApplication.instance() or QtWidgets.QApplication([])

        self._build_tracking_window()
        self._build_data_window()
        self._build_2d_window()

        self.timer = QtCore.QTimer()
        self.timer.timeout.connect(self._tick)
        self.timer.start(UPDATE_MS)

# -----
# Window 1 — Tracking
# -----

    def _build_tracking_window(self):

        def style_plot(plot, tick_size=12, title_size=13):
            for axis in ['left', 'bottom']:
                plot.getAxis(axis).setStyle(tickFont=QtGui.QFont('Arial', tick_size))
            plot.titleLabel.item.setFont(
                QtGui.QFont('Arial', title_size, QtGui.QFont.Weight.Bold))

        self.tracking_win = QtWidgets.QWidget()
        self.tracking_win.setWindowTitle("Ball Tracker — 3D Position")
        self.tracking_win.resize(1200, 620)
        self.tracking_win.setStyleSheet("background-color: #1a1a2e;")

        outer = QtWidgets.QHBoxLayout()
        outer.setContentsMargins(4, 4, 4, 4)
        outer.setSpacing(8)
        self.tracking_win.setLayout(outer)

        # ---- Left: 3D GL view embedded in window ----
        self.gl_widget = gl.GLViewWidget()

```

```

self.gl_widget.setCameraPosition(distance=12, elevation=25, azimuth=45)
self.gl_widget.setBackgroundColor("#cdcdf1")
self.gl_widget.setMinimumSize(600, 500)
outer.addWidget(self.gl_widget, stretch=4)

# Build 3D scene inside the embedded widget
self.view3d = Visualiser3D(widget=self.gl_widget)

# ---- Centre: IMU orientation cube ----
self.imu_gl = gl.GLViewWidget()
# Change distance from 3 to 4 to give the cube more 'breathing room'
self.imu_gl.setCameraPosition(distance=4, elevation=25, azimuth=45)
self.imu_gl.setBackgroundColor("#2e2e42")
self.imu_gl.setMinimumSize(200, 200)
self.imu_gl.setMaximumWidth(240)
outer.addWidget(self.imu_gl, stretch=1)

# IMU cube axes for orientation reference
self.imu_cube = gl.GLBoxItem(size=QtGui.QVector3D(1.0, 0.6, 0.4))
# Use a bright, solid color to ensure it's not blending into the white background
self.imu_cube.setColor((0.2, 0.5, 1.0, 1.0))

# Add the item to the GL widget
self.imu_gl.addItem(self.imu_cube)

# Axis lines on IMU cube — X=red, Y=green, Z=blue
for pts, col in [
    (np.array([[0,0,0],[0.8,0,0]]), (1,0,0,1)),
    (np.array([[0,0,0],[0,0.5,0]]), (0,1,0,1)),
    (np.array([[0,0,0],[0,0,0.4]]), (0.3,0.5,1,1)),
]:
    ax = gl.GLLinePlotItem(pos=pts, color=col, width=3, antialias=True)
    self.imu_gl.addItem(ax)

self._imu_roll = 0.0
self._imu_pitch = 0.0
self._imu_yaw = 0.0

# Kick flash placeholder (used by _tick for flash timer)
self.flash_timer = 0

# ---- Right: speedometer + stats ----
right_widget = QtWidgets.QWidget()
right_widget.setStyleSheet("background-color: #1a1a2e;")
right_layout = QtWidgets.QVBoxLayout()

```

```

right_layout.setContentsMargins(4, 4, 4, 4)
right_layout.setSpacing(8)
right_widget.setLayout(right_layout)
outer.addWidget(right_widget, stretch=2)

self.speedo = Speedometer()
right_layout.addWidget(self.speedo, stretch=3)

# Stats panel
self.stats_panel = QtWidgets.QFrame()
self.stats_panel.setStyleSheet("""
    QFrame {
        background-color: #0d0d1f;
        border: 2px solid #4a90d9;
        border-radius: 10px;
    }
""")
stats_layout = QtWidgets.QVBoxLayout()
stats_layout.setContentsMargins(14, 10, 14, 10)
stats_layout.setSpacing(6)
self.stats_panel.setLayout(stats_layout)

title_lbl = QtWidgets.QLabel("LIVE STATS")
title_lbl.setStyleSheet(
    "color: #4a90d9; font-size: 12px; font-weight: bold; "
    "letter-spacing: 2px; border: none;")
title_lbl.setAlignment(QtCore.Qt.AlignmentFlag.AlignCenter)
stats_layout.addWidget(title_lbl)

divider = QtWidgets.QFrame()
divider.setFrameShape(QtWidgets.QFrame.Shape.HLine)
divider.setStyleSheet("background-color: #4a90d9; border: none; max-height: 1px;")
stats_layout.addWidget(divider)

def make_stat_row(title, value_color="#ffffff"):
    row = QtWidgets.QHBoxLayout()
    t = QtWidgets.QLabel(title)
    t.setStyleSheet("color: #8888bb; font-size: 11px; border: none;")
    v = QtWidgets.QLabel("")
    v.setStyleSheet(
        f"color: {value_color}; font-size: 14px; font-weight: bold; border: none;")
    v.setAlignment(QtCore.Qt.AlignmentFlag.AlignRight)
    row.addWidget(t)
    row.addWidget(v)
    stats_layout.addLayout(row)

```

```

        return v

    self.stat_pos = make_stat_row("X / Y", "#00e5ff")
    self.stat_height = make_stat_row("Height Z", "#ff9800")
    self.stat_speed = make_stat_row("Speed m/s", "#ffffff")
    self.stat_kmh = make_stat_row("Speed km/h", "#64b5f6")
    self.stat_vz = make_stat_row("Vert. vel", "#ce93d8")
    self.stat_spin = make_stat_row("Spin", "#f48fb1")
    self.stat_r1 = make_stat_row("Range r1", "#ef5350")
    self.stat_r2 = make_stat_row("Range r2", "#66bb6a")
    self.stat_r3 = make_stat_row("Range r3", "#26c6da")
    self.stat_r4 = make_stat_row("Range r4", "#ffd54f")
    self.stat_kicks = make_stat_row("Kicks", "#ff7043")

    right_layout.addWidget(self.stats_panel, stretch=4)

    self.tracking_win.show()

# -----
# Window 2 — Sensor Data
# -----

def _build_data_window(self):

    def style_plot(plot):
        font_tick = QtGui.QFont('Arial', 14, QtGui.QFont.Weight.Bold)
        font_title = QtGui.QFont('Arial', 16)
        font_label = QtGui.QFont('Arial', 13)
        for axis in ['left', 'bottom']:
            plot.getAxis(axis).setStyle(tickFont=font_tick)
            plot.getAxis(axis).setTextPen(pg.mkPen('#1a1a2e'))
            plot.getAxis(axis).label.setFont(font_label)
            # Fix applied here:
            plot.getAxis(axis).label.setDefaultTextColor(QtGui.QColor('#1a1a2e'))
        plot.titleLabel.item.setFont(font_title)
        plot.titleLabel.item.setDefaultTextColor(QtGui.QColor('#1a1a2e'))

    self.data_win = pg.GraphicsLayoutWidget(show=True,
                                             title="Ball Tracker — Sensor Data")
    self.data_win.resize(1200, 620)
    self.data_win.setBackground("#ffffff")

    # Row 1 — accel + gyro side by side
    self.accel_plot = self.data_win.addPlot(title="Acceleration (m/s²)")

```

```

style_plot(self.accel_plot)

self.accel_plot.addLegend()
self.accel_plot.showGrid(x=True, y=True, alpha=0.3)
self.accel_plot.setYRange(-ACCEL_Y_RANGE, ACCEL_Y_RANGE)
self.accel_plot.setLabel('left', 'm/s2')
self.ax_buf = deque(maxlen=BUFFER_LEN)
self.ay_buf = deque(maxlen=BUFFER_LEN)
self.az_buf = deque(maxlen=BUFFER_LEN)
self.ax_curve = self.accel_plot.plot(pen=pg.mkPen('r', width=2), name='ax')
self.ay_curve = self.accel_plot.plot(pen=pg.mkPen('c', width=2), name='ay')
self.az_curve = self.accel_plot.plot(pen=pg.mkPen('y', width=2), name='az')
self.accel_plot.addItem(pg.InfiniteLine(
    pos=KICK_THRESHOLD, angle=0,
    pen=pg.mkPen('r', width=1, style=QtCore.Qt.PenStyle.DashLine)
))

self.gyro_plot = self.data_win.addPlot(title="Gyroscope (rad/s)")
style_plot(self.gyro_plot)

self.gyro_plot.addLegend()
self.gyro_plot.showGrid(x=True, y=True, alpha=0.3)
self.gyro_plot.setYRange(-20, 20)
self.gyro_plot.setLabel('left', 'rad/s')
self.gx_buf = deque(maxlen=BUFFER_LEN)
self.gy_buf = deque(maxlen=BUFFER_LEN)
self.gz_buf = deque(maxlen=BUFFER_LEN)
self.gx_curve = self.gyro_plot.plot(pen=pg.mkPen("#fbff00", width=2), name='gx')
self.gy_curve = self.gyro_plot.plot(pen=pg.mkPen("#00c0e2", width=2), name='gy')
self.gz_curve = self.gyro_plot.plot(pen=pg.mkPen("#ff3300", width=2), name='gz')

# Row 2 – ranges + speed side by side
self.data_win.nextRow()
self.range_plot = self.data_win.addPlot(title="UWB Ranges (m)")
style_plot(self.range_plot)

self.range_plot.addLegend()
self.range_plot.showGrid(x=True, y=True, alpha=0.3)
self.range_plot.setYRange(0, RANGE_Y_MAX)
self.range_plot.setLabel('left', 'metres')
self.r1_buf = deque(maxlen=BUFFER_LEN)
self.r2_buf = deque(maxlen=BUFFER_LEN)
self.r3_buf = deque(maxlen=BUFFER_LEN)
self.r4_buf = deque(maxlen=BUFFER_LEN)
self.r1_curve = self.range_plot.plot(pen=pg.mkPen('r', width=2), name='r1')

```

```

self.r2_curve = self.range_plot.plot(pen=pg.mkPen('g', width=2), name='r2')
self.r3_curve = self.range_plot.plot(pen=pg.mkPen('c', width=2), name='r3')
self.r4_curve = self.range_plot.plot(pen=pg.mkPen('#ffd54f', width=1), name='r4')

self.speed_plot = self.data_win.addPlot(title="Speed over Time (m/s)")
style_plot(self.speed_plot)

self.speed_plot.showGrid(x=True, y=True, alpha=0.3)
self.speed_plot.setYRange(0, SPEED_Y_MAX)
self.speed_plot.setLabel('left', 'm/s')
self.speed_buf = deque(maxlen=BUFFER_LEN)
self.speed_curve = self.speed_plot.plot(pen=pg.mkPen('y', width=3))

# -----
# Window 3 — 2D Top-Down
# -----

def _build_2d_window(self):
    def style_plot(plot, tick_size=12, title_size=13):
        for axis in ['left', 'bottom']:
            plot.getAxis(axis).setStyle(tickFont=QtGui.QFont('Arial', tick_size))
        plot.titleLabel.item.setFont(
            QtGui.QFont('Arial', title_size, QtGui.QFont.Weight.Bold))

    self.win_2d = pg.GraphicsLayoutWidget(show=True,
                                           title="Ball Tracker — Top-Down 2D")
    self.win_2d.resize(600, 600)
    self.win_2d.setBackground("#e1f1ca")

    self.pos_plot = self.win_2d.addPlot(title="Ball Position (Top-Down X/Y)")
    self.pos_plot.setAspectLocked(True)
    self.pos_plot.showGrid(x=True, y=True, alpha=0.3)
    self.pos_plot.setLabel('left', 'Y (m)')
    self.pos_plot.setLabel('bottom', 'X (m)')
    self.pos_plot.enableAutoRange()

    # Anchor markers
    anchor_colours = ['b', 'b', 'b', 'b']
    for i, (anchor, col) in enumerate(zip([A1, A2, A3, A4], anchor_colours)):
        self.pos_plot.plot([anchor[0]], [anchor[1]],
                           pen=None, symbol='s', symbolSize=12,
                           symbolBrush=col)
        lbl = pg.TextItem(
            text=f"A{i+1}\n({'gnd' if anchor[2] == 0 else f'z={anchor[2]}m'})",
            color='black', anchor=(0.5, 1.5))

```

```

    )
    lbl.setPos(anchor[0], anchor[1])
    self.pos_plot.addItem(lbl)

# Trail + ball dot
self.x_trail = deque(maxlen=TRAIL_LENGTH)
self.y_trail = deque(maxlen=TRAIL_LENGTH)
self.trail_curve = self.pos_plot.plot(pen=pg.mkPen('b', width=2))
self.tag_dot = self.pos_plot.plot(pen=None, symbol='o',
                                   symbolSize=18, symbolBrush='y')

# Velocity arrow
self.vel_arrow = pg.ArrowItem(angle=0, tipAngle=30, headLen=20, tailLen=20,
                               pen=pg.mkPen('g', width=2), brush='g')
self.pos_plot.addItem(self.vel_arrow)

# Kick flash ring
self.kick_flash = self.pos_plot.plot(pen=None, symbol='o', symbolSize=44,
                                       symbolBrush=None,
                                       symbolPen=pg.mkPen('r', width=3))
self.kick_flash.setVisible(False)
self.flash_timer_2d = 0

# -----
# Tick
# -----

def _tick(self):
    result = self.on_update()

    if result is None:
        return

# Unpack 3D result
x, y, z, vx, vy, vz, ax, ay, az, gx, gy, gz, r1, r2, r3, r4, kick, kick_count = result

speed_3d = math.sqrt(vx**2 + vy**2 + vz**2)
speed_2d = math.sqrt(vx**2 + vy**2)
spin = math.sqrt(gx**2 + gy**2 + gz**2) # rad/s

# ---- 3D view ----
self.view3d.update(x, y, z, vx, vy, vz, kick, kick_count, spin)

# ---- 2D window ----
self.x_trail.append(x)

```

```

self.y_trail.append(y)
self.trail_curve.setData(list(self.x_trail), list(self.y_trail))
self.tag_dot.setData([x], [y])

self.vel_arrow.setPos(x, y)
if speed_2d > 0.05:
    angle = math.degrees(math.atan2(vy, vx))
    self.vel_arrow.setStyle(angle=180 - angle)

if kick:
    self.kick_flash.setData([x], [y])
    self.kick_flash.setVisible(True)
    self.flash_timer_2d = KICK_FLASH_MS

if self.flash_timer_2d > 0:
    self.flash_timer_2d -= UPDATE_MS
    if self.flash_timer_2d <= 0:
        self.kick_flash.setVisible(False)

# ---- Speedometer ----
self.speedo.set_speed(speed_3d)

# ---- Stats panel ----
self.stat_pos.setText(f"{x:.2f}, {y:.2f} m")
self.stat_height.setText(f"{z:.2f} m")
self.stat_speed.setText(f"{speed_3d:.2f} m/s")
self.stat_kmh.setText(f"{speed_3d * 3.6:.1f} km/h")
self.stat_vz.setText(f"{vz:+.2f} m/s")
self.stat_spin.setText(f"{spin:.1f} rad/s")
self.stat_r1.setText(f"{r1:.2f} m")
self.stat_r2.setText(f"{r2:.2f} m")
self.stat_r3.setText(f"{r3:.2f} m")
self.stat_r4.setText(f"{r4:.2f} m")
self.stat_kicks.setText(str(kick_count))

# ---- Data window ----
self.ax_buf.append(ax)
self.ay_buf.append(ay)
self.az_buf.append(az)
self.ax_curve.setData(list(self.ax_buf))
self.ay_curve.setData(list(self.ay_buf))
self.az_curve.setData(list(self.az_buf))

self.gx_buf.append(gx)
self.gy_buf.append(gy)

```

```

self.gz_buf.append(gz)
self.gx_curve.setData(list(self.gx_buf))
self.gy_curve.setData(list(self.gy_buf))
self.gz_curve.setData(list(self.gz_buf))

self.r1_buf.append(r1)
self.r2_buf.append(r2)
self.r3_buf.append(r3)
self.r4_buf.append(r4)
self.r1_curve.setData(list(self.r1_buf))
self.r2_curve.setData(list(self.r2_buf))
self.r3_curve.setData(list(self.r3_buf))
self.r4_curve.setData(list(self.r4_buf))

self.speed_buf.append(speed_3d)
self.speed_curve.setData(list(self.speed_buf))

# ---- IMU orientation cube ----
# Integrate gyro to estimate orientation angles
# 1. Update the accumulation variables
dt = UPDATE_MS / 1000.0
self._imu_roll += gx * dt * (180.0 / math.pi)
self._imu_pitch += gy * dt * (180.0 / math.pi)
self._imu_yaw += gz * dt * (180.0 / math.pi)

# 2. CLEAR previous frame's transformations
self.imu_cube.resetTransform()

# 3. APPLY rotations in a stable order (Yaw -> Pitch -> Roll)
self.imu_cube.rotate(self._imu_yaw, 0, 0, 1) # Rotate around Z
self.imu_cube.rotate(self._imu_pitch, 0, 1, 0) # Rotate around Y
self.imu_cube.rotate(self._imu_roll, 1, 0, 0) # Rotate around X

# 4. CENTER the cube AFTER rotating it
# This ensures it spins in place in the middle of the side panel
self.imu_cube.translate(-0.5, -0.3, -0.2)

def run(self):
    import sys
    sys.exit(self.app.exec())

```

7.3.7 Visualiser_3d.py

```

# =====
# visualiser_3d.py — 3D OpenGL tracking window
# =====
#
# Shows:
# - Ball trail (coloured line)
# - Ball sphere
# - Spinning cube indicating ball spin rate
# - Anchor positions (labelled cubes)
# - Anchor bounding cuboid (wireframe)
# - Velocity arrow
# - Kick flash (red ring)
# - Ground plane grid
# =====

import math
import numpy as np
from collections import deque

import pyqtgraph as pg
import pyqtgraph.opengl as gl
from pyqtgraph.Qt import QtCore, QtGui, QtWidgets

from config import (
    A1, A2, A3, A4,
    TRAIL_LENGTH, KICK_FLASH_MS, UPDATE_MS
)

class Visualiser3D:
    def __init__(self, widget=None):
        if widget is not None:
            self.win = widget
            self._owns_window = False
        else:
            self.win = gl.GLViewWidget()
            self.win.setWindowTitle("Ball Tracker — 3D View")
            self.win.resize(900, 700)
            self.win.setCameraPosition(distance=10, elevation=25, azimuth=45)
            self.win.setBackgroundColor("#d3f7b6")
            self.win.show()
            self._owns_window = True

        self._build_ground_grid()
        self._build_anchor_cuboid()

```

```

self._build_anchors()
self._build_trail()
self._build_ball()
self._build_spin_cube()
self._build_velocity_arrow()
self._build_kick_flash()

self.flash_timer = 0
self._spin_angle = 0.0
self._ball_pos = np.array([0.0, 0.0, 0.0])

# -----
# Ground grid
# -----

def _build_ground_grid(self):
    anchors = [A1, A2, A3, A4]
    xs = [a[0] for a in anchors]
    ys = [a[1] for a in anchors]
    cx = (max(xs) + min(xs)) / 2
    cy = (max(ys) + min(ys)) / 2
    span = max(max(xs)-min(xs), max(ys)-min(ys)) + 3.0

    grid = gl.GLGridItem()
    grid.setSize(span, span)
    grid.setSpacing(1, 1)
    grid.translate(cx, cy, 0)
    grid.setColor((60, 60, 100, 120))
    self.win.addItem(grid)

# -----
# Anchor bounding cuboid
# -----

def _build_anchor_cuboid(self):
    """
    Draw a wireframe box whose corners are at the XY extents
    of the anchors, with Z going from 0 (floor) to max anchor height.
    """
    anchors = [A1, A2, A3, A4]
    xs = [a[0] for a in anchors]
    ys = [a[1] for a in anchors]
    zs = [a[2] for a in anchors]

    x0, x1 = min(xs), max(xs)

```

```

y0, y1 = min(ys), max(ys)
z0, z1 = 0.0, max(zs)      # floor to highest anchor

# 8 corners of the box
corners = np.array([
    [x0, y0, z0], [x1, y0, z0],
    [x1, y1, z0], [x0, y1, z0],
    [x0, y0, z1], [x1, y0, z1],
    [x1, y1, z1], [x0, y1, z1],
], dtype=float)

# 12 edges as pairs of corner indices
edges = [
    (0,1),(1,2),(2,3),(3,0), # bottom face
    (4,5),(5,6),(6,7),(7,4), # top face
    (0,4),(1,5),(2,6),(3,7), # verticals
]

for a, b in edges:
    line = gl.GLLinePlotItem(
        pos=np.array([corners[a], corners[b]]),
        color=(0.0, 0.0, 0.0, 1),
        width=1,
        antialias=True
    )
    self.win.addItem(line)

# -----
# Anchors
# -----

def _build_anchors(self):
    anchors = [A1, A2, A3, A4]
    colours = [
        (255, 80, 80, 255),
        (255, 80, 80, 255),
        (255, 80, 80, 255),
        (255, 80, 80, 255),
    ]
    for i, (anchor, col) in enumerate(zip(anchors, colours)):
        x, y, z = anchor

        cube = gl.GLBoxItem(size=QtGui.QVector3D(0.15, 0.15, 0.15))
        cube.setColor(col)
        cube.translate(x - 0.075, y - 0.075, z - 0.075)

```

```

self.win.addItem(cube)

line_pts = np.array([[x, y, 0], [x, y, z]], dtype=float)
line = gl.GLLinePlotItem(
    pos=line_pts,
    color=(1.0, 1.0, 1.0, 0.25),
    width=1,
    antialias=True
)
self.win.addItem(line)

try:
    label = gl.GLTextItem(
        pos=np.array([x, y, z + 0.2]),
        text=f"A{i+1}",
        color=QtGui.QColor(*col)
    )
    self.win.addItem(label)
except AttributeError:
    pass

# -----
# Trail
# -----

def _build_trail(self):
    self.trail_x = deque(maxlen=TRAIL_LENGTH)
    self.trail_y = deque(maxlen=TRAIL_LENGTH)
    self.trail_z = deque(maxlen=TRAIL_LENGTH)

    self.trail_item = gl.GLLinePlotItem(
        pos=np.zeros((2, 3)),
        width=3,
        antialias=True,
        mode='line_strip'
    )
    self.win.addItem(self.trail_item)

def _trail_colour_array(self, n):
    colours = np.zeros((n, 4), dtype=float)
    for i in range(n):
        alpha = i / max(n - 1, 1)
        colours[i] = [0.2, 255, 255, alpha * 0.9]
    return colours

```

```

# -----
# Ball sphere
# -----

def _build_ball(self):
    sphere = gl.MeshData.sphere(rows=12, cols=12, radius=0.15)
    self.ball_mesh = gl.GLMeshItem(
        meshdata=sphere,
        smooth=True,
        color=(1.0, 0.85, 0.1, 0),
        shader='shaded',
        glOptions='opaque'
    )
    self.win.addItem(self.ball_mesh)

# -----
# Spin cube – small transparent wireframe that rotates
# at a rate proportional to gyro magnitude
# -----

def _build_spin_cube(self):
    self.spin_cube = gl.GLBoxItem(
        size=QtGui.QVector3D(0.22, 0.22, 0.22)
    )
    self.spin_cube.setColor((0.9, 0.4, 1.0, 0.35))
    self.win.addItem(self.spin_cube)
    self._spin_angle = 0.0

# -----
# Velocity arrow
# -----

def _build_velocity_arrow(self):
    self.arrow_item = gl.GLLinePlotItem(
        pos=np.zeros((2, 3)),
        color=(0.0, 1.0, 0.4, 1.0),
        width=4,
        antialias=True
    )
    self.win.addItem(self.arrow_item)
    self.arrow_item.setVisible(False)

# -----
# Kick flash ring
# -----

```

```

def _build_kick_flash(self):
    n = 32
    theta = np.linspace(0, 2 * math.pi, n)
    ring = np.zeros((n, 3))
    ring[:, 0] = 0.4 * np.cos(theta)
    ring[:, 1] = 0.4 * np.sin(theta)

    self.kick_ring = gl.GLLinePlotItem(
        pos=ring,
        color=(1.0, 0.2, 0.2, 1.0),
        width=3,
        antialias=True,
        mode='line_strip'
    )
    self.win.addItem(self.kick_ring)
    self.kick_ring.setVisible(False)
    self._kick_pos = np.array([0.0, 0.0, 0.0])

# -----
# Update
# -----

def update(self, x, y, z, vx, vy, vz, kick, kick_count, spin=0.0):
    bz = max(0.0, z)
    speed = math.sqrt(vx**2 + vy**2 + vz**2)

    # ---- Trail ----
    self.trail_x.append(x)
    self.trail_y.append(y)
    self.trail_z.append(bz)
    n = len(self.trail_x)
    if n >= 2:
        pts = np.column_stack([
            list(self.trail_x),
            list(self.trail_y),
            list(self.trail_z)
        ])
        self.trail_item.setData(pos=pts, color=self._trail_colour_array(n))

    # ---- Ball + spin cube — move to ball position ----
    dx = x - self._ball_pos[0]
    dy = y - self._ball_pos[1]
    dz = bz - self._ball_pos[2]
    self.ball_mesh.translate(dx, dy, dz)

```

```

# Spin cube: reset transform, move to ball, apply rotation
self.spin_cube.resetTransform()
# Rotate around Z axis at spin rate — angle accumulates each tick
self._spin_angle += spin * (UPDATE_MS / 1000.0) * (180.0 / math.pi)
self._spin_angle %= 360.0
self.spin_cube.rotate(self._spin_angle, 0, 0, 1)
self.spin_cube.translate(x - 0.11, y - 0.11, bz - 0.11)

self._ball_pos = np.array([x, y, bz])

# ---- Velocity arrow ----
if speed > 0.1:
    scale = min(speed * 0.3, 1.5)
    end = np.array([x + vx/speed*scale,
                    y + vy/speed*scale,
                    bz + vz/speed*scale])
    self.arrow_item.setData(pos=np.array([x, y, bz], end))
    self.arrow_item.setVisible(True)
else:
    self.arrow_item.setVisible(False)

# ---- Kick flash ----
if kick:
    self._kick_pos = np.array([x, y, bz])
    self.flash_timer = KICK_FLASH_MS

if self.flash_timer > 0:
    self.flash_timer -= UPDATE_MS
    theta = np.linspace(0, 2 * math.pi, 32)
    ring = np.zeros((32, 3))
    ring[:, 0] = self._kick_pos[0] + 0.4 * np.cos(theta)
    ring[:, 1] = self._kick_pos[1] + 0.4 * np.sin(theta)
    ring[:, 2] = self._kick_pos[2]
    self.kick_ring.setData(pos=ring)
    self.kick_ring.setVisible(True)
if self.flash_timer <= 0:
    self.kick_ring.setVisible(False)

```

7.3.7 Simulate.py

```

# =====
# simulate.py — 3D fake data for testing without hardware
# =====

import math
import random
from position_filter import PositionFilter
from kick_detector import KickDetector
from visualiser import Visualiser
from trilaterate import trilaterate
import config

t = 0.0

# Orbit centre based on actual anchor positions
cx = (config.A1[0] + config.A2[0] + config.A3[0] + config.A4[0]) / 4
cy = (config.A1[1] + config.A2[1] + config.A3[1] + config.A4[1]) / 4

def fake_ranges_3d(x, y, z, noise=0.02):
    anchors = [config.A1, config.A2, config.A3, config.A4]
    ranges = []
    for a in anchors:
        d = math.sqrt((x-a[0])**2 + (y-a[1])**2 + (z-a[2])**2)
        ranges.append(d + random.gauss(0, noise))
    return tuple(ranges)

class SimApp:
    def __init__(self):
        self.pos_filter = PositionFilter()
        self.kick_det = KickDetector()
        self.vis = Visualiser(on_update=self.process)

    def process(self):
        global t
        t += 0.05

        # Ball orbits the centre of the anchor layout
        x_raw = cx + 1.5 * math.cos(t * 0.5)
        y_raw = cy + 1.5 * math.sin(t * 0.5)
        z_raw = max(0.0, 0.4 * abs(math.sin(t * 1.5)))

        r1, r2, r3, r4 = fake_ranges_3d(x_raw, y_raw, z_raw)

```

```

ax = random.gauss(0, 0.5)
ay = random.gauss(0, 0.5)
az = 9.81 + random.gauss(0, 0.3)
# Simulate spin — faster when moving
speed = math.sqrt((x_raw-config.A1[0])**2 + (y_raw-config.A1[1])**2)
gx = random.gauss(0, 1.0) * speed
gy = random.gauss(0, 1.0) * speed
gz = random.gauss(0, 2.0) * speed
kick = random.random() < 0.005

pos = trilaterate(config.A1, config.A2, config.A3, config.A4,
                  r1, r2, r3, r4)
if pos is None:
    print("[SIM] trilaterate returned None — check anchor coords in config.py")
    return None

uwb_x, uwb_y, uwb_z = pos
x, y, z, vx, vy, vz = self.pos_filter.update(uwb_x, uwb_y, uwb_z)

if kick:
    self.kick_det.kick_count += 1

return (x, y, z, vx, vy, vz,
        ax, ay, az,
        gx, gy, gz,
        r1, r2, r3, r4,
        kick, self.kick_det.kick_count)

def run(self):
    self.vis.run()

if __name__ == "__main__":
    SimApp().run()

```